编译原理
13. 垃圾回收

rainoftime.github.io
浙江大学
计算机科学与技术学院

Content

Outline

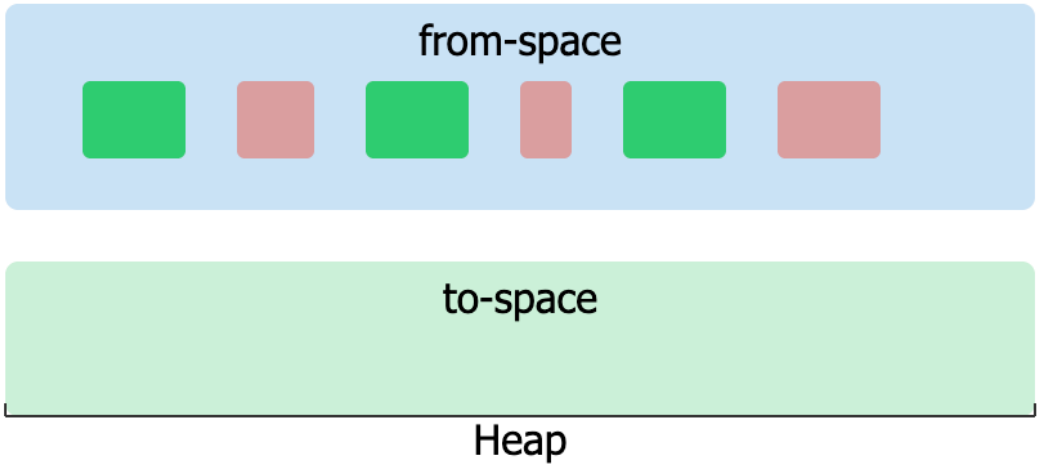
1. Introduction
2. Mark-and-Sweep
3. Reference Count
4. **Copying Collection**
5. **Interface to the Compiler**

3. Copying Collection

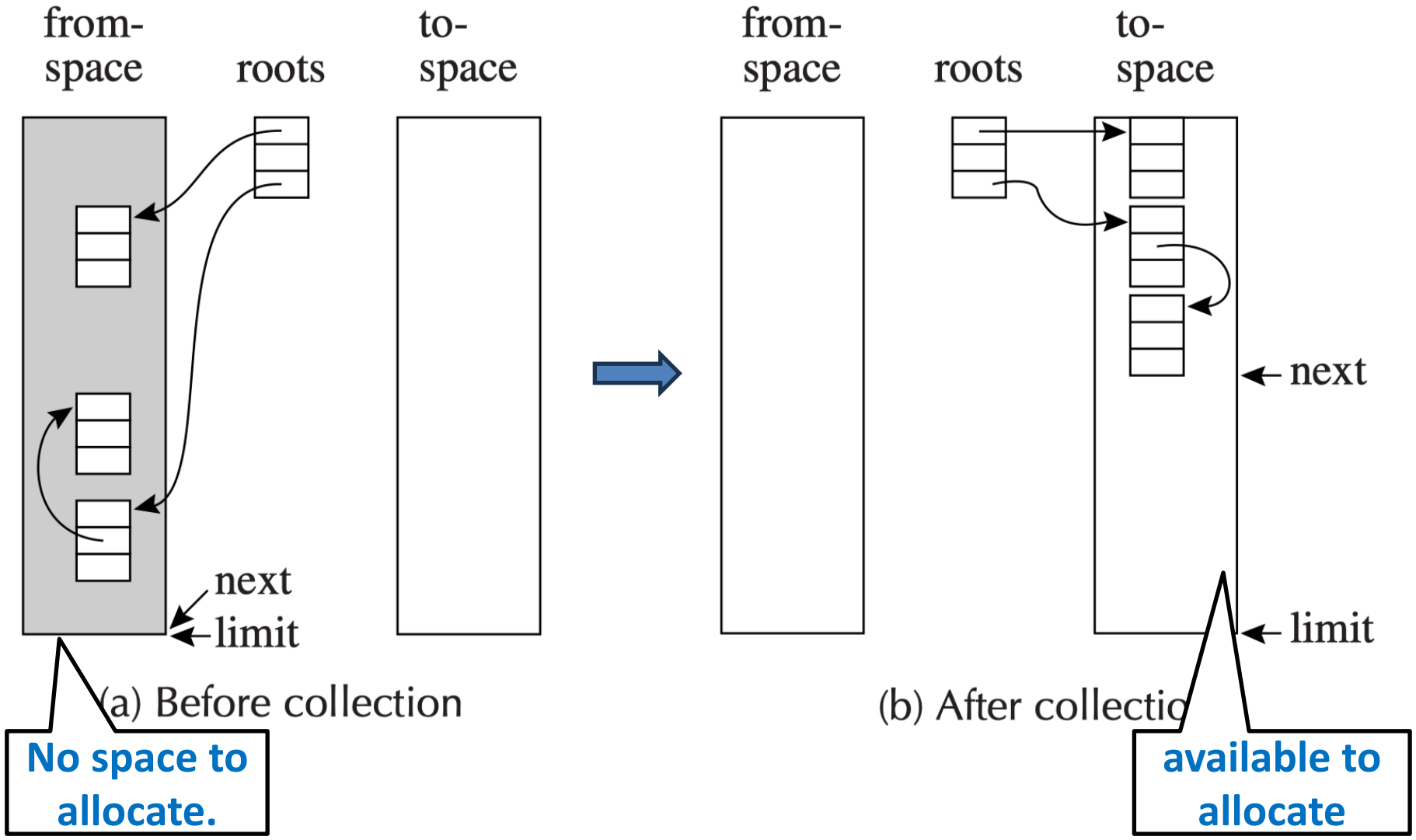
- The Basic Idea
- Cheney's Algorithm
- Pointer Forwarding
- Locality & Improvements

Copying Collection

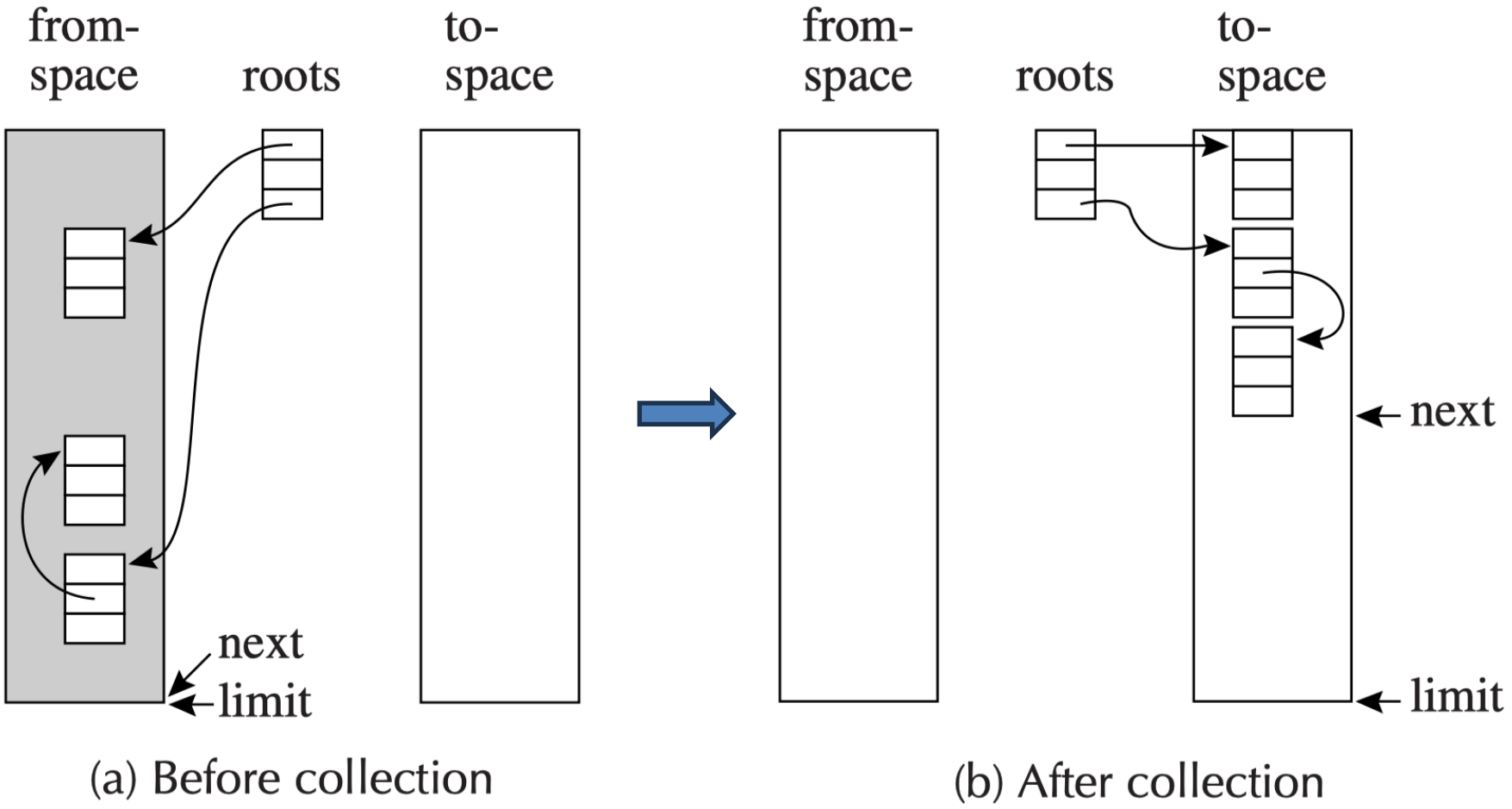
- **Memory division: use 2 heaps**
  - from-space: the one used by program
  - to-space: the one unused until GC time
- When from-space fills up:
  1. Copy all **reachable** objects to to-space
  2. **Swap the roles** of the two spaces



Copying Collection



Copying Collection

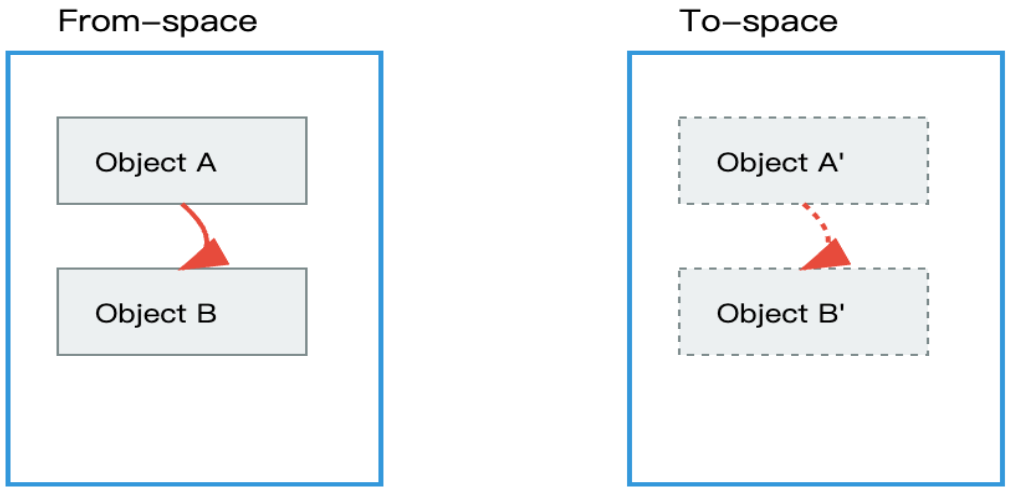


- **Very fast allocation** (just increment pointer): p=next, next=next + n.
- **No fragmentation**: All the reachable nodes are around the beginning of the to-space

Challenges in Copying Collection

- Copying is *not* a simple memcpy — the object graph has structure that must be preserved:

1. **Copy all reachable objects (as for mark-and-sweep)**
2. **Preserve all pointer relationships between objects**
3. **Avoid copying the same object multiple times**



Why Not Simple Recursion

- A naive recursive copy (DFS) seems natural:

```
function Copy(p):
    new_p = allocate in to-space
    for each field f_i of p:
        new_p.f_i = Copy(p.f_i)    // recurse
    return new_p
```

- Problems:
  - **Stack depth ∝ longest pointer chain**
  - No mechanism to detect already-copied objects → **infinite loops on cycles!**
  - **How to detect** already-copied objects

3. Copying Collection

- The Basic Idea
- Cheney's Algorithm
- Pointer Forwarding
- Locality & Improvements

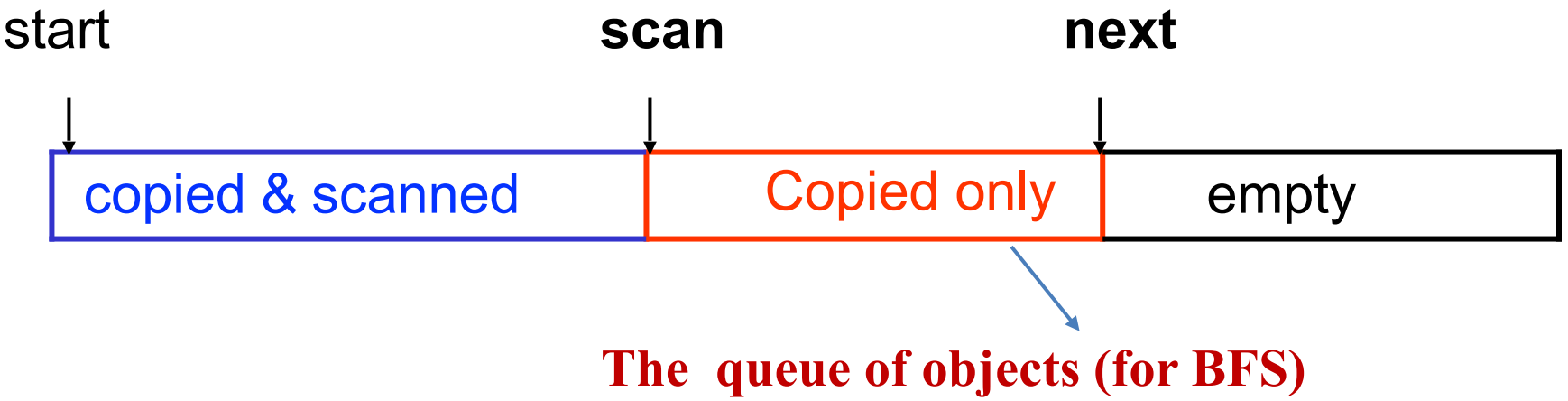
The Challenge of Recursive Traversal

- The drawback of simple recursive copying:
  - Requires stack space proportional to the length of the longest pointer chain
  - Stack overflow for deeply nested data structures

- **Cheney's Algorithm**: Using a Work Queue in To-space
  - Similar to to pointer reversal trick for DFS
  - Avoids additional storage for the BFS queue

Cheney's Algorithm (BFS Coping)

- Partition the **to-space** in three contiguous regions
  - Copied: the record is copied, but we haven't yet looked at pointers inside the record
  - Copied and scanned: the record is copied, and we have processed all pointers in the record



When **scan** catches up to **next**, all reachable objects have been copied and scanned!

Cheney's Algorithm (BFS Coping)

Pointer forwarning

```
function Forward(p)
    if p is a pointer to from-space then
        if p.f1 points to to-space then
            return p.f1
        else
            for each field fi of p
                next.fi ← p.fi
            p.f1 ← next
            next ← next +size of record p
            return p->f1
    else
        return p
```

Cheney's algorithm

```
scan ← next ← beginning of to-space
for each root r    // forward all roots
    r ← Forward(r)
while scan < next
    for each field fi at scan
        scan.fi ← Forward(scan.fi)
    scan ← scan + size of record at scan
```

主循环：第一阶段对所有根调用 Forward（入队）；第二阶段循环处理队列：对 scan 处的对象的每个字段调用 Forward（更新指针并可能入队新对象），然后推进 scan。

Forward(p)（后面再展开）

1. 检查 p 是否在 from-space若 p.f1 已指向 to-space，说明已复制，直接返回转发指针；
2. 若 p.f1 不是 to-space 地址，把 p 的所有字段复制到 next 位置，在原对象设置转发指针，next 后移，返回新地址；
3. 若 p 不在 from-space，返回 p 不变（已经是 to-space 地址或是整数等非指针值）。

Cheney's Algorithm (BFS Coping)

Pointer forwarning

```
function Forward(p)
    if p is a pointer to from-space then
        if p.f1 points to to-space then
            return p.f1
        else
            for each field fi of p
                next.fi ← p.fi
            p.f1 ← next
            next ← next +size of record p
            return p->f1
    else
        return p
```

Cheney's algorithm

```
scan ← next ← beginning of to-space
for each root r    // forward all roots
    r ← Forward(r)
while scan < next
    for each field fi at scan
        scan.fi ← Forward(scan.fi)
    scan ← scan + size of record at scan
```

scan和next指针之间的区域作为工作队列

1. 当对象被复制时加入队列（向前移动next）
2. 当对象被扫描时从队列中移除（向前移动scan）
3. 当scan追上next时，队列为空，遍历完成

- scan指针指向下一个需要被扫描处理的对象: 在**to-space中、已经被复制但其内部指针字段还没有被处理**的对象。
- next指针指向to-space中的空闲位置，也就是下一个要被复制到to-space的对象将存放的位置。

Cheney's Algorithm

```
scan ← next ← beginning of to-space
for each root r
    r ← Forward(r)                          // forward all roots
while scan < next                           // process scan queue
    for each field fi of record at scan
        scan.fi ← Forward(scan.fi)          // Forward each field
    scan ← scan + size of record at scan    // Move to next record
```

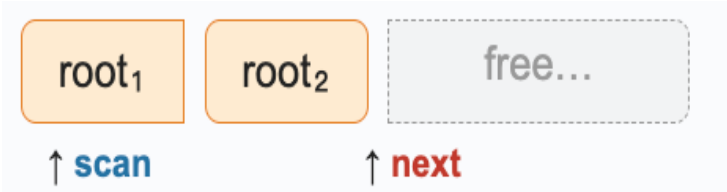
1. **Initialize scan to the beginning of to-space**
2. Copy all roots to to-space (advancing next)
3. While scan < next:
   - For each pointer field in the object at scan:
     - Apply the Forward function
     - Update the field to point to the to-space copy
   - Advance scan to the next object

Cheney's Algorithm

```
scan ← next ← beginning of to-space
for each root r
    r ← Forward(r)                    // forward all roots
while scan < next                     // process scan queue
    for each field fi of record at scan
        scan.fi ← Forward(scan.fi)    // Forward each field
    scan ← scan + size of record at scan  // Move to next record
```

2. Copy all roots to to-space (advancing next)

- Root objects are copied to to-space, next advances
- After this phase: **scan** = start, **next** = after last root object
- The BFS queue now contains root objects (unscanned)



These objects' fields still point to from-space!

Cheney's Algorithm

```
scan ← next ← beginning of to-space
for each root r
    r ← Forward(r)                    // forward all roots
while scan < next                     // process scan queue
    for each field fi of record at scan
        scan.fi ← Forward(scan.fi)    // Forward each field
    scan ← scan + size of record at scan  // Move to next record
```

3. Scan the queue

- Take the object at **scan** (front of queue)
- For each of its pointer field, call Forward
  - If the target is **not yet copied** → copies it to next, queue grows
  - If **already copied** → just returns the forwarding pointer
- Update the field to point to the to-space copy
- Advance scan to the next object

Cheney's Algorithm

```
scan ← next ← beginning of to-space
for each root r
      r ← Forward(r)                    // forward all roots
while scan < next                       // process scan queue
      for each field fi of record at scan
            scan.fi ← Forward(scan.fi)        // Forward each field
      scan ← scan + size of record at scan  // Move to next record
```

3. Scan the queue

- Take the object at **scan** (front of queue)
- For each of its pointer field, call Forward
  - If the target is **not yet copied** → copies it to next, queue grows
  - If **already copied** → just returns the forwarding pointer
- Update the field to point to the to-space copy
- Advance scan to the next object

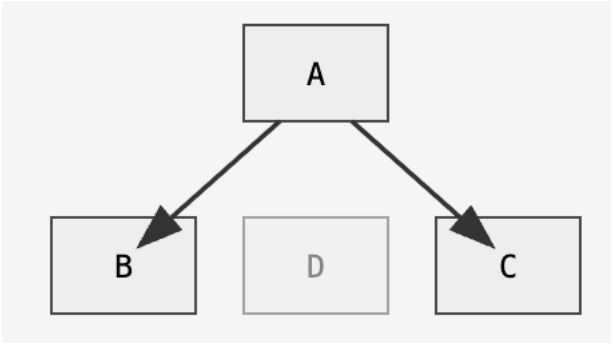
Cheney's Algorithm

```
scan ← next ← beginning of to-space
for each root r
    r ← Forward(r)                          // forward all roots
while scan < next                           // process scan queue
    for each field fi of record at scan
        scan.fi ← Forward(scan.fi)          // Forward each field
    scan ← scan + size of record at scan    // Move to next record
```

When scan catches up to next,
all reachable objects have been copied and scanned!

Example: Initial State

- **A** is the root; it has two pointer fields → B and C
- **B** and **C** have no outgoing references
- **D** is unreachable from the root



From-space

A:
→ ptr to B
→ ptr to C

B:
(no pointers)

C:
(no pointers)

D:
(unreachable)

To-space

(Empty)

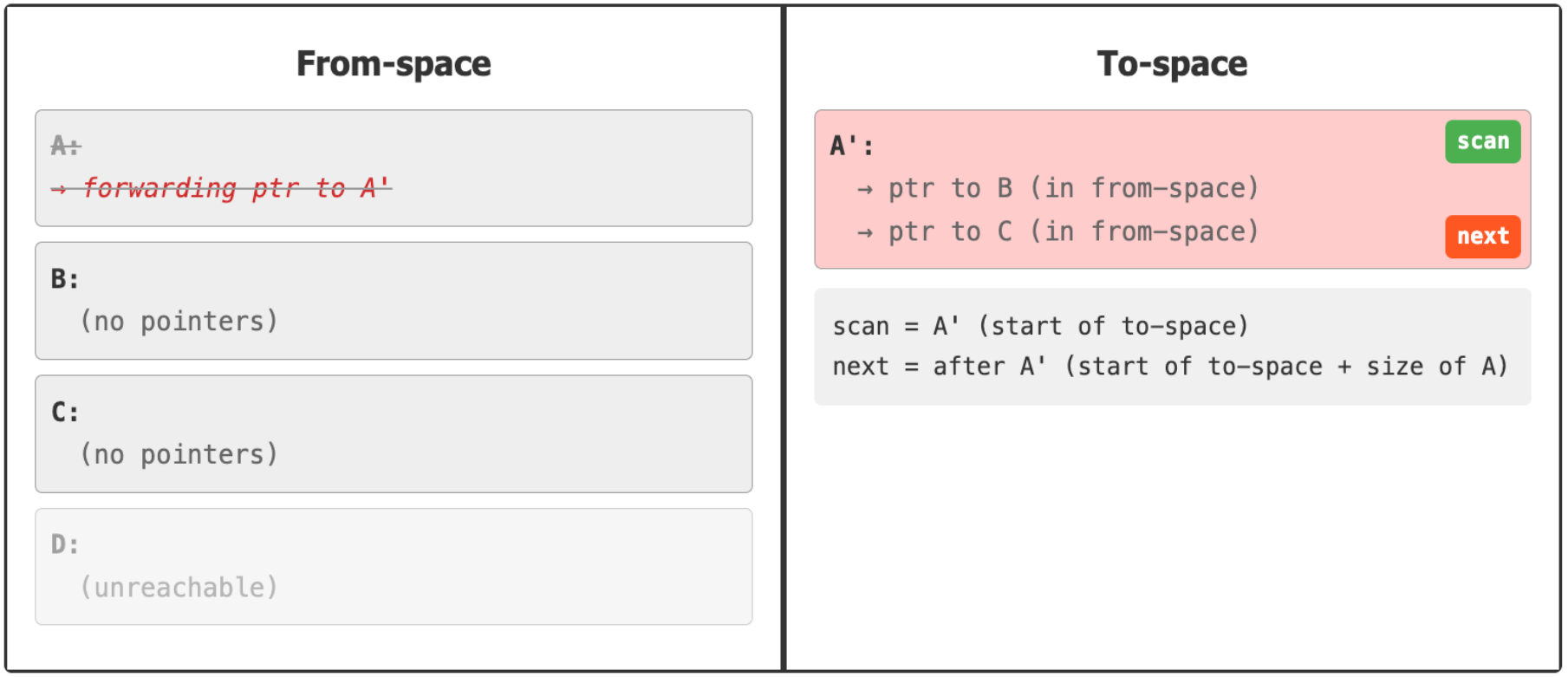
scan ↓ start

next ↓ start

```
scan = next = start of to-space
```

Step 1: Forward Root A

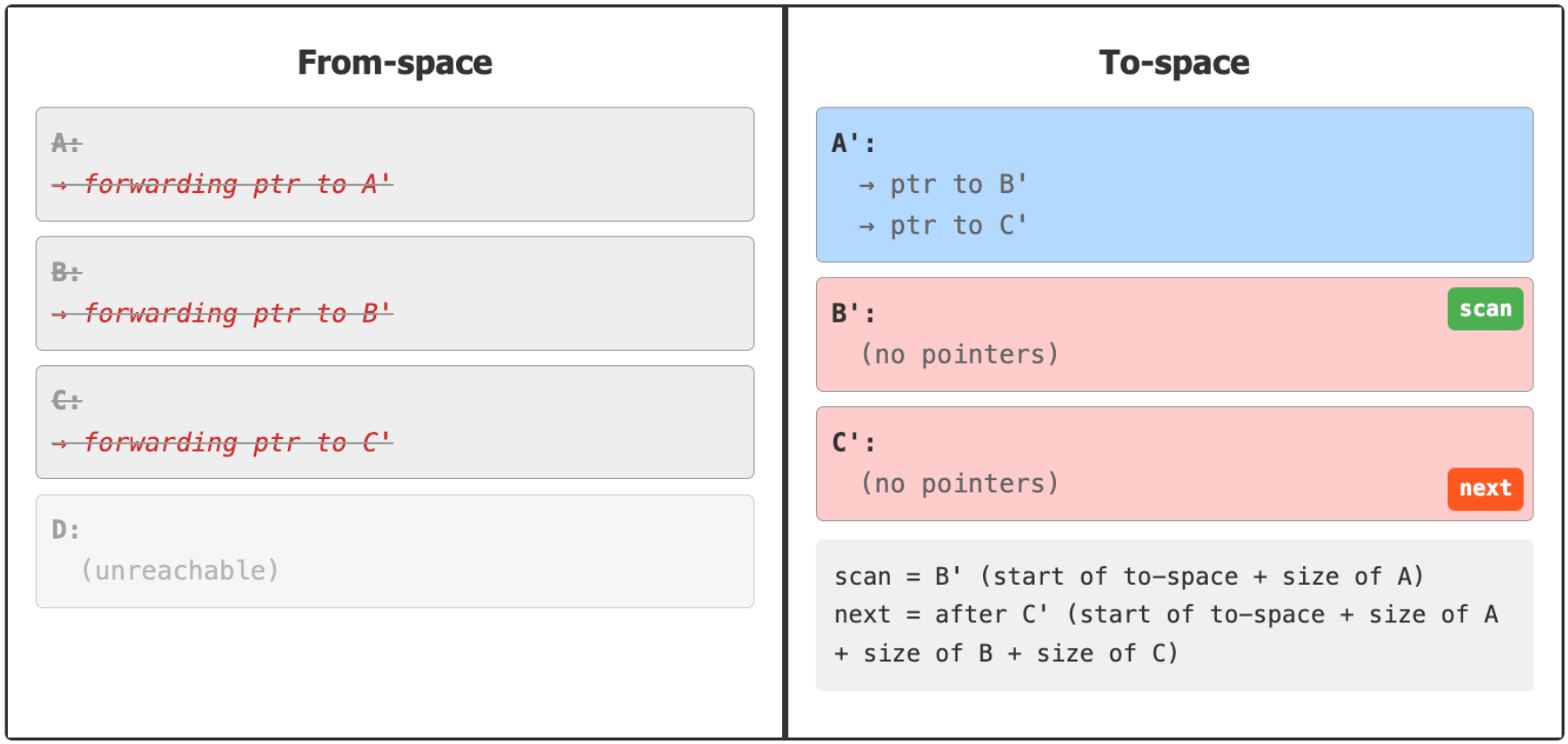
- First, we copy the root object (A) to to-space and leave a forwarding pointer in the original object.



A' is copied but not yet scanned (its fields still hold the *original* from-space addresses of B and C.)

Step 2: Scan A' – Forward Its Pointer Fields

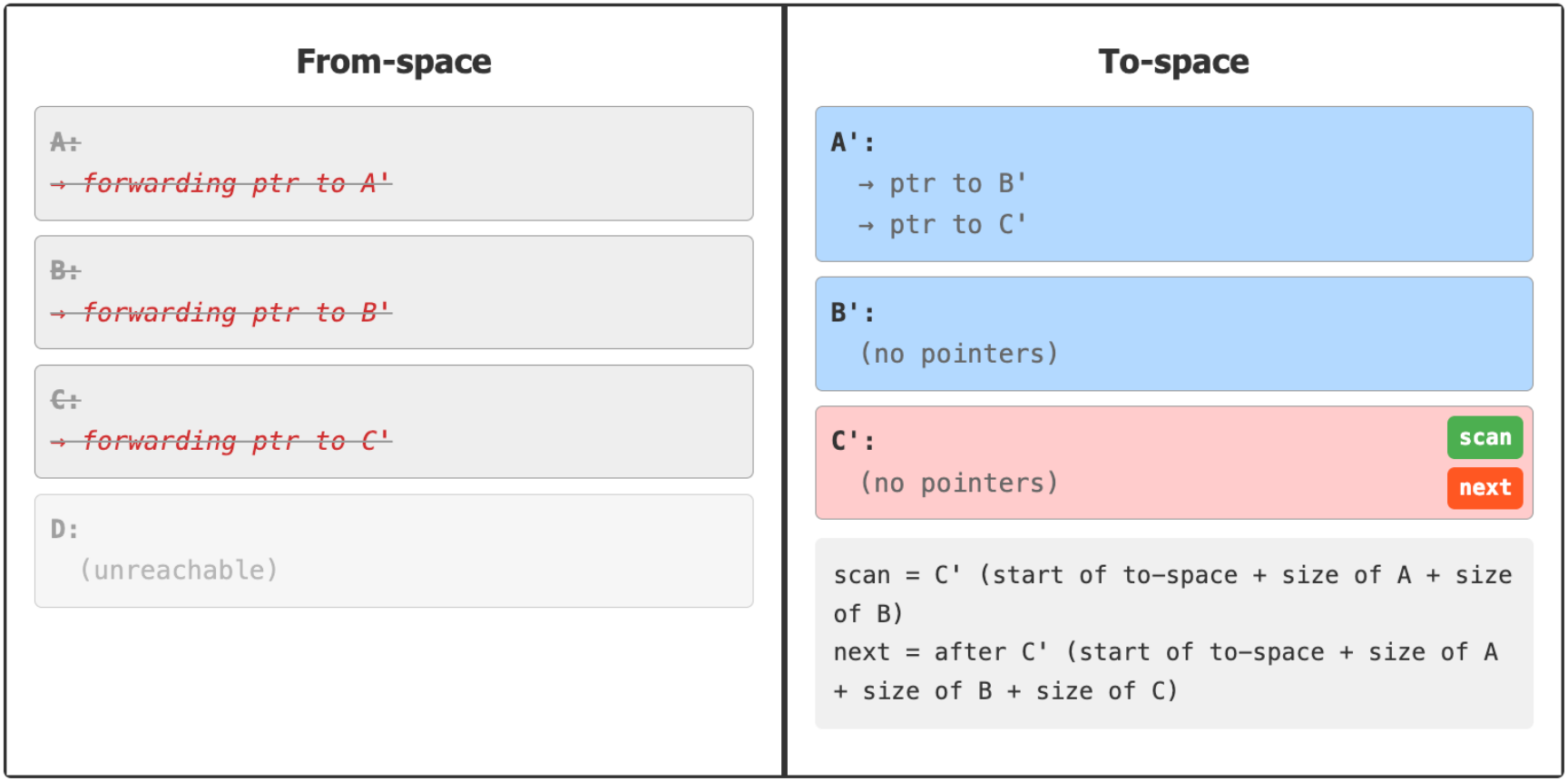
- while scan < next: scan is at A' (process the object at scan (A') by forwarding its pointers to to-space.)



A' is now copied and scanned.
B' and C' are copied but not yet scanned.

Step 3: Scan B', then C'

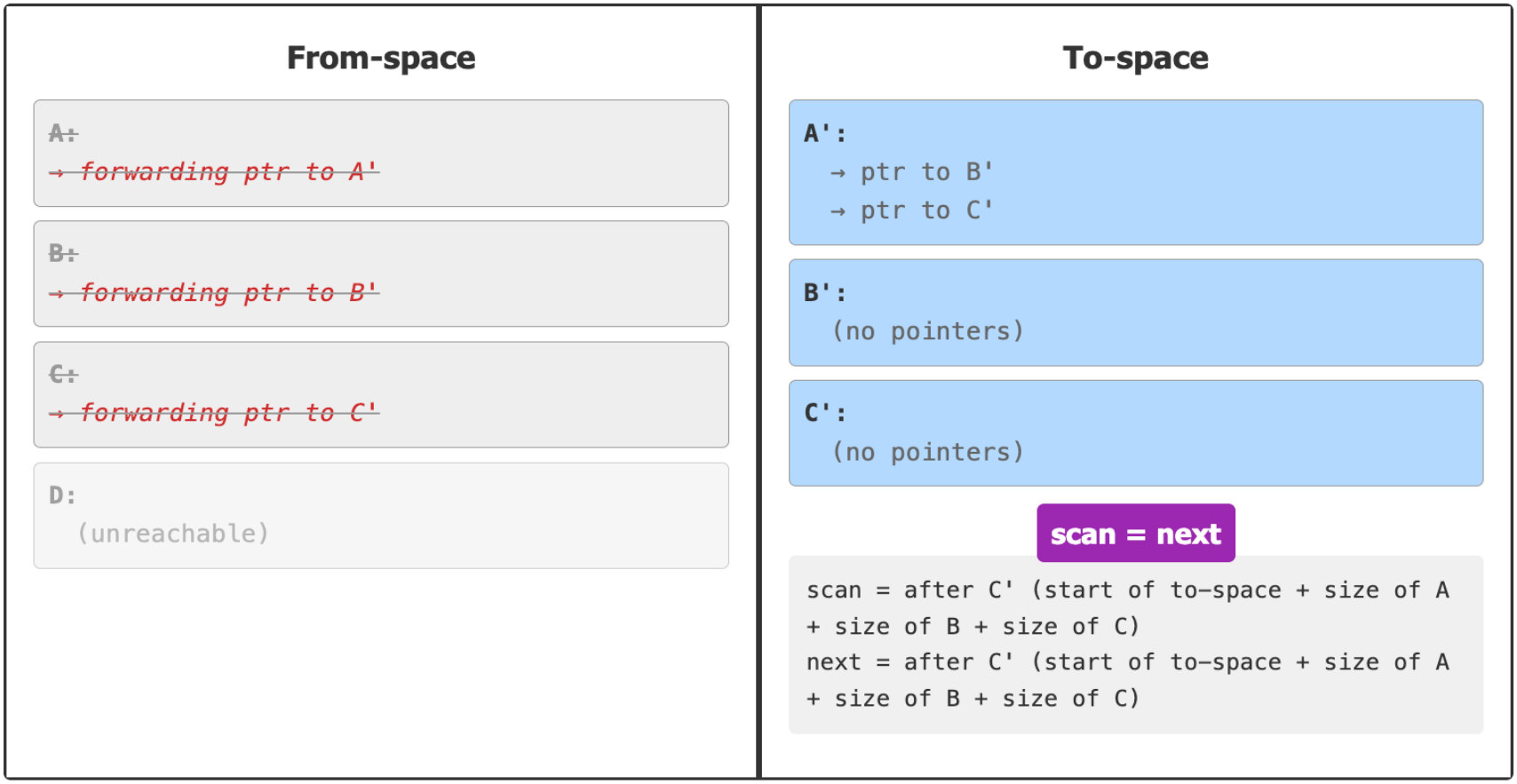
- Now we process the object at scan (B') by forwarding its pointers to to-space.



B' is now copied and scanned.
C' is copied but not yet scanned.

Step 3: Scan B', then C'

- Now we process the object at scan (C') by forwarding its pointers to to-space.



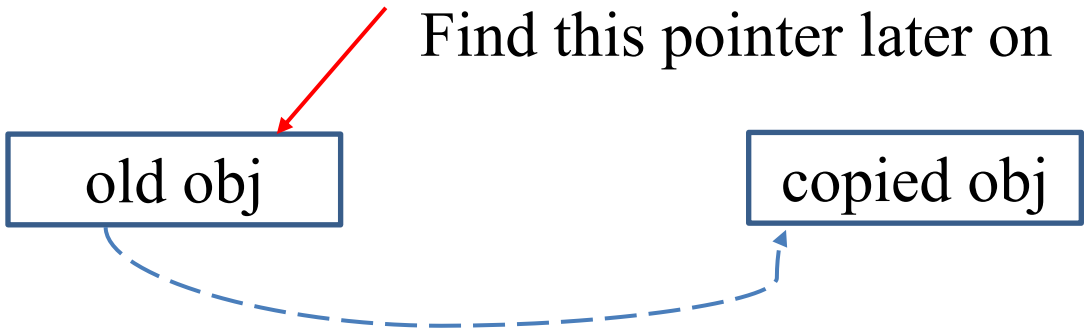
At this point, scan = next, which means we've processed all reachable objects and the collection is complete!

3. Copying Collection

- The Basic Idea
- Cheney's Algorithm
- Pointer Forwarding
- Locality & Improvements

Pointer Forwarding

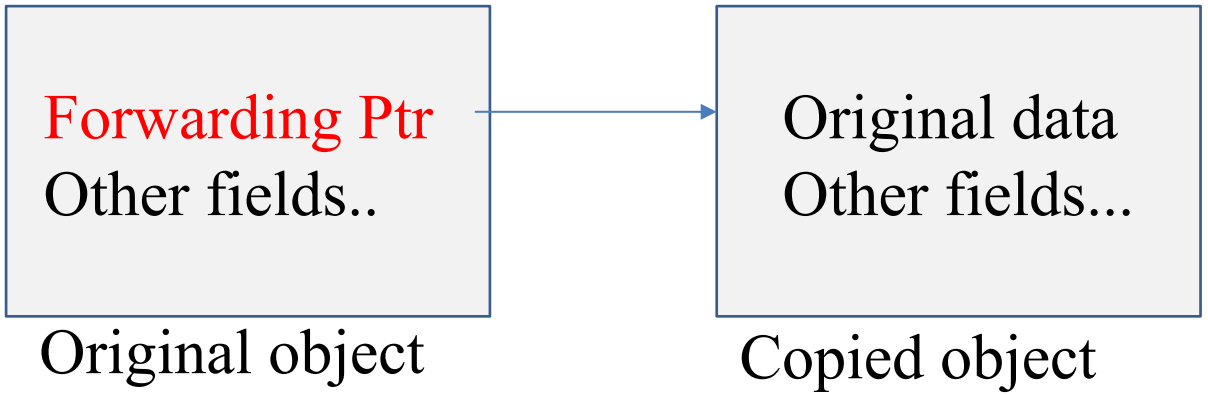
- Basic operation/sub-procedure in copying collection
- Main goals of pointer forwarding
  1. Locate an object's new position
  2. Ensure all pointers correctly reference the new copies
  3. Ensure each object is copied exactly once



Pointer Forwarding

- **forwarding pointer:** After copying an object from from-space to to-space, **overwrite** the first field ($p.f_1$) of the *original* object with a pointer to the *new copy*

  1. "Has this object been copied?" → Check if $p.f_1$ points into to-space
  2. "Where is the new copy?" → Just follow $p.f_1$



Three Cases at a Glance

- **Case I : Already Copied: p ∈ from-space AND $p.f_1$ ∈ to-space**
  - p was encountered and copied earlier. Its first field $p.f_1$ was overwritten with a forwarding pointer to the to-space copy.
  - Action: Return $p.f_1$ directly. No copy needed. This ensures shared objects have exactly one copy.
- **Case II: Not yet copied p ∈ from-space AND $p.f_1$ ∉ to-space**
  - First encounter of this object. Copy all fields to next in to-space; overwrite $p.f_1 \leftarrow$ next (install forwarding pointer); advance next; return $p.f_1$.
- **Case III: Not a from-space pointer** p ∉ from-space
  - p is already in to-space, or not a heap pointer at all
  - Action: Return p unchanged

Pointer Forwarding: Case I

```
function Forward(p)
    if p points to from-space then
        if p. f1 points to to-space then        // Case I: Already copied
            return p. f1                        // Return forwarding pointer
        else                                    // Case II: not yet copied
            for each field fi of p
                next. fi ← p. fi                // Copy fields to to-space
            p. f1 ← next                        // Install forwarding pointer
            next ← next + size of record p      // Advance to-space pointer
            return p. f1                        // Return address in to-space
    else                                // Already in to-space or not a pointer
        return p
```

1. **Already copied:** p points to a from-space record that has already been copied
   - Return the forwarding pointer (p.f1) to the to-space copy

Pointer Forwarding: Case II

```
function Forward(p)
    if p points to from-space then
        if p. f1 points to to-space then        // Case I: Already copied
            return p. f1                        // Return forwarding pointer
        else                                    // Case II: not yet copied
            for each field fi of p
                next. fi ← p. fi                // Copy fields to to-space
            p. f1 ← next                        // Install forwarding pointer
            next ← next + size of record p      // Advance to-space pointer
            return p. f1                        // Return address in to-space
    else                                // Already in to-space or not a pointer
        return p
```

1. Already copied: p points to a from-space record already copied
2. **Not yet copied**: p points to an object without a forwarding pointer
   - Copy the entire object to to-space
   - Install a forwarding pointer in the original object
   - Return the pointer to the new copy

Pointer Forwarding: Case III

```
function Forward(p)
    if p points to from-space then
        if p. f1 points to to-space then        // Already copied
            return p. f1                        // Return forwarding pointer
        else
            for each field fi of p
                next. fi ← p. fi                // Copy fields to to-space
            p. f1 ← next                        // Install forwarding pointer
            next ← next + size of record p      // Advance to-space pointer
            return p. f1                        // Return address in to-space
    else                                // Case III: Already in to-space or not a pointer
        return p
```

1. Already copied: p points to a from-space record already copied
2. Not yet copied: p points to an object without a forwarding pointer
3. **Not a from-space pointer:** already in to-space or isn't a pointer
   - Return p unchanged

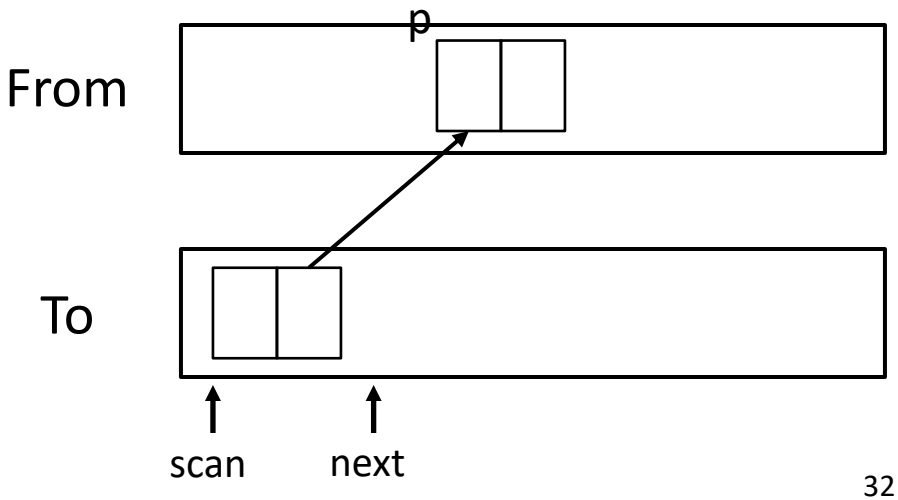
Example: Pointer Forwarding

```
function Forward(p)
    if p is a pointer to from-space then
        if p.f1 points to to-space then
            return p.f1
        else
            for each field fi of p
                next.fi ← p.fi
            p.f1 ← next
            next ← next +size of record p
            return p->f1
    else
        return p
```

Cheney's algorithm

```
scan ← next ← beginning of to-space
for each root r
    r ← Forward(r)
while scan < next
    for each field fi at scan
        scan.fi ← Forward(scan.fi)
    scan ← scan + size of record at scan
```

假设我们到达了右图的状态。
并且算法正在执行scan.f_i ← Forward(scan.f_i) 这一行。



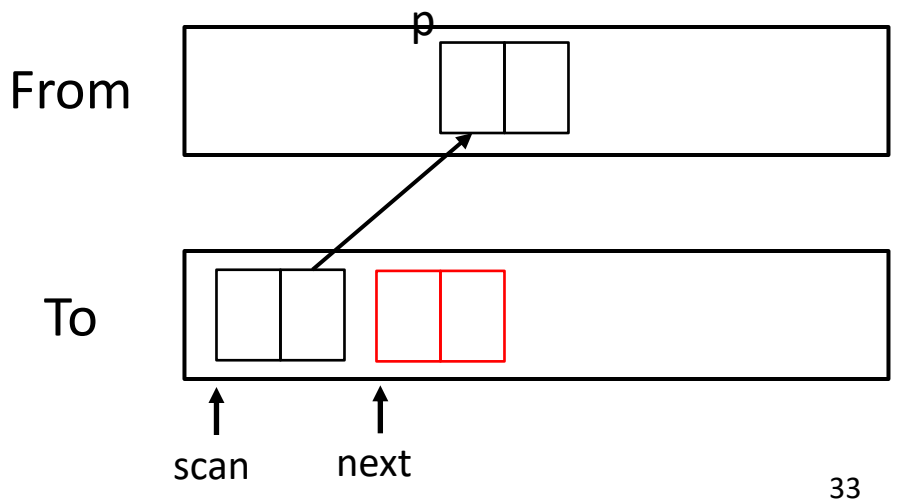
Example: Pointer Forwarding

```
function Forward(p)
    if p is a pointer to from-space then
        if p.f1 points to to-space then
            return p.f1
        else
            for each field fi of p
                next.fi ← p.fi
            p.f1 ← next
            next ← next +size of record p
            return p->f1
    else
        return p
```

```
scan ← next ← beginning of to-space
for each root r
    r ← Forward(r)
while scan < next
    for each field fi at scan
        scan.fi ← Forward(scan.fi)
    scan ← scan + size of record at scan
```

p.f1还没有指向to-space(Case II)

1. 复制: 通过Forward函数里的这个循环，把p指向的对象的每个field 都复制

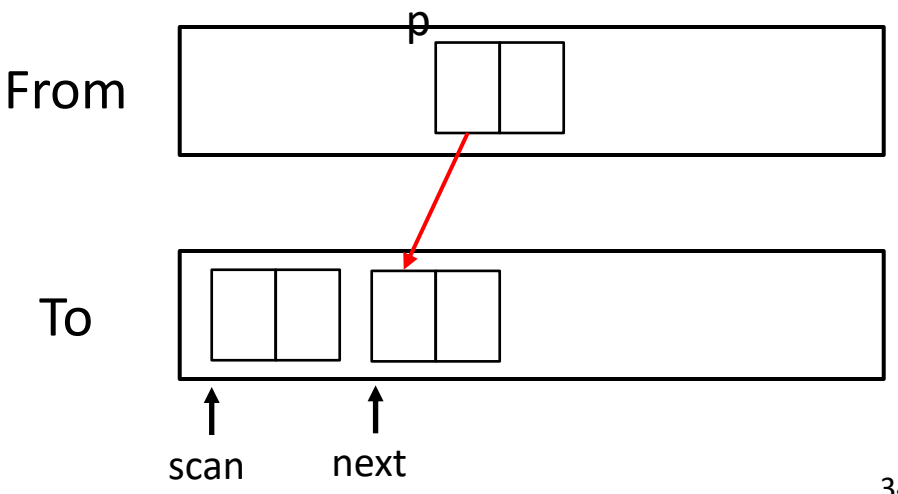


Example: Pointer Forwarding

```
function Forward(p)
    if p is a pointer to from-space then
        if p.f1 points to to-space then
            return p.f1
        else
            for each field fi of p
                next.fi ← p.fi
            p.f1 ← next
            next ← next +size of record p
            return p->f1
    else
        return p
```

```
scan ← next ← beginning of to-space
for each root r
        r ← Forward(r)
 while scan < next
        for each field fi at scan
            scan.fi ← Forward(scan.fi)
        scan ← scan + size of record at scan
```

2. 把p.f1设为forwarding point
(指向to-space中被复制的对象)

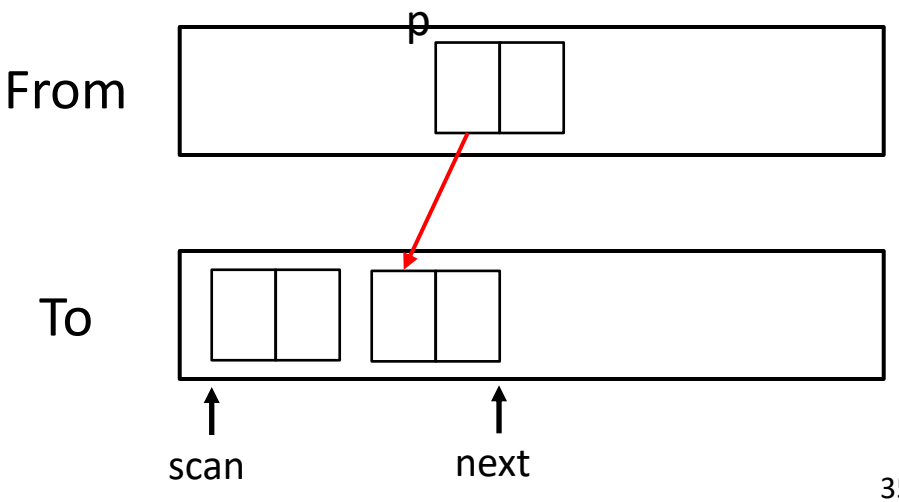


Example: Pointer Forwarding

```
function Forward(p)
    if p is a pointer to from-space then
        if p.f1 points to to-space then
            return p.f1
        else
            for each field fi of p
                next.fi ← p.fi
            p.f1 ← next
            next ← next +size of record p
            return p->f1
    else
        return p
```

```
scan ← next ← beginning of to-space
for each root r
        r ← Forward(r)
 while scan < next
        for each field fi at scan
            scan.fi ← Forward(scan.fi)
        scan ← scan + size of record at scan
```

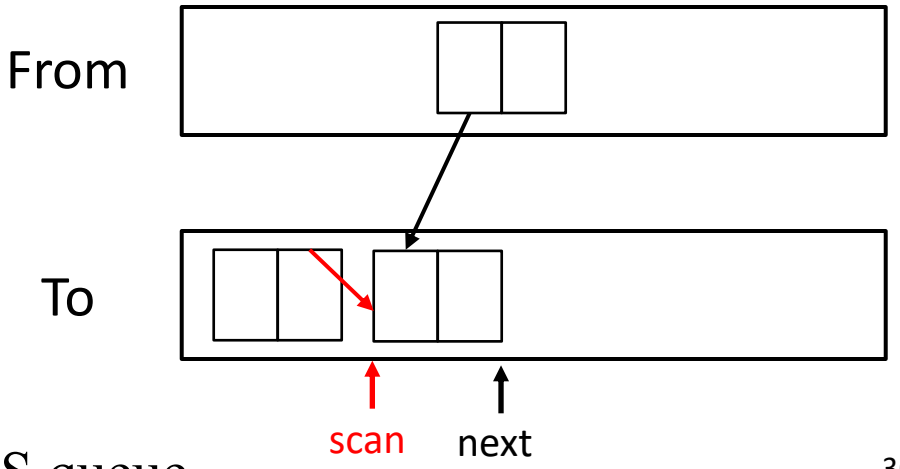
3. 最后，移动next指针，并返回p->f1



Example: Pointer Forwarding

```
function Forward(p)
    if p is a pointer to from-space then
        if p.f1 points to to-space then
            return p.f1
        else
            for each field fi of p
                next.fi ← p.fi
            p.f1 ← next
            next ← next +size of record p
            return p->f1
    else
        return p
```

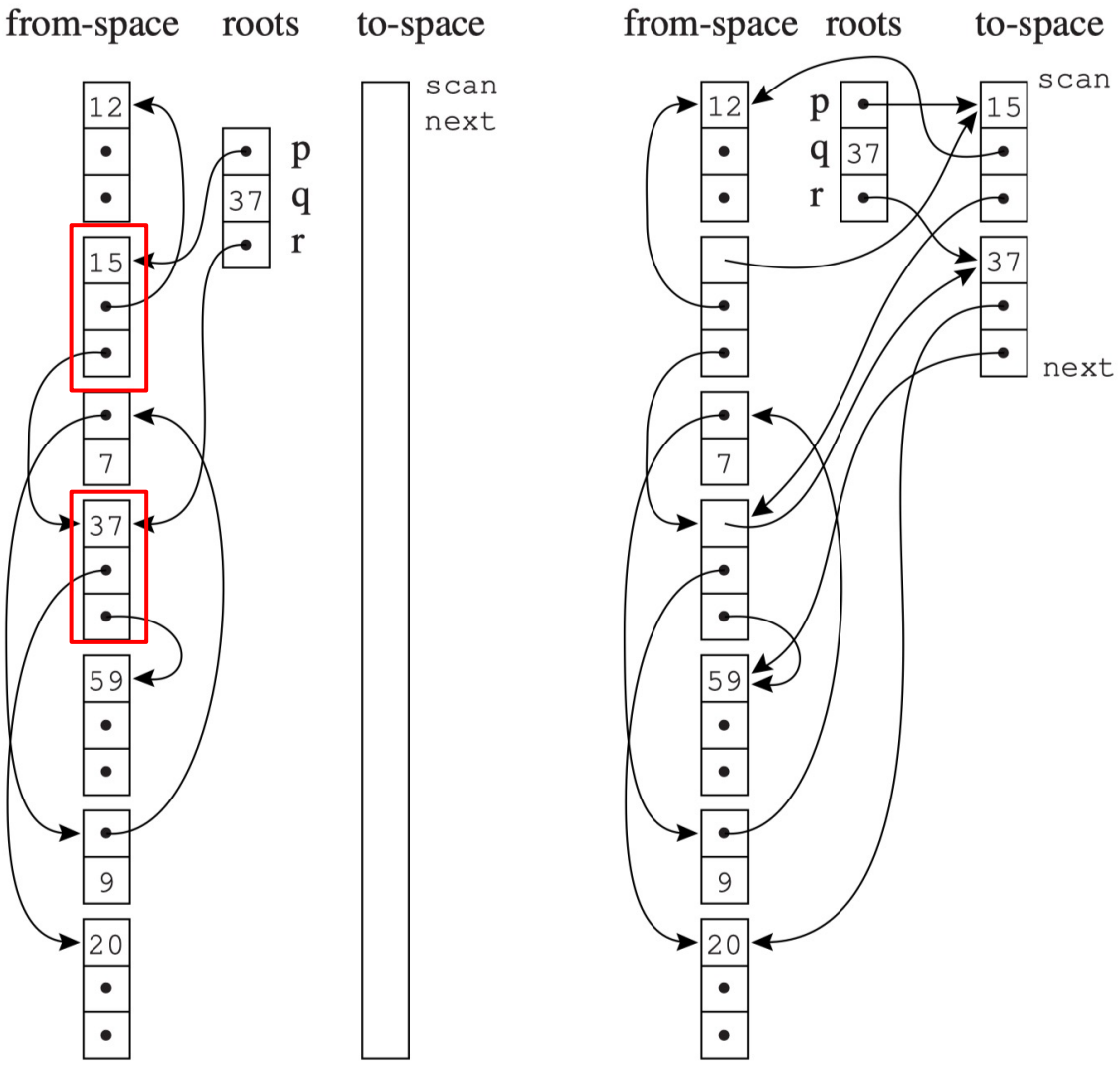
```
scan ← next ← beginning of to-space
for each root r
    r ← Forward(r)
while scan < next
    for each field fi at scan
        scan.fi ← Forward(scan.fi)
    scan ← scan + size of record at scan
```



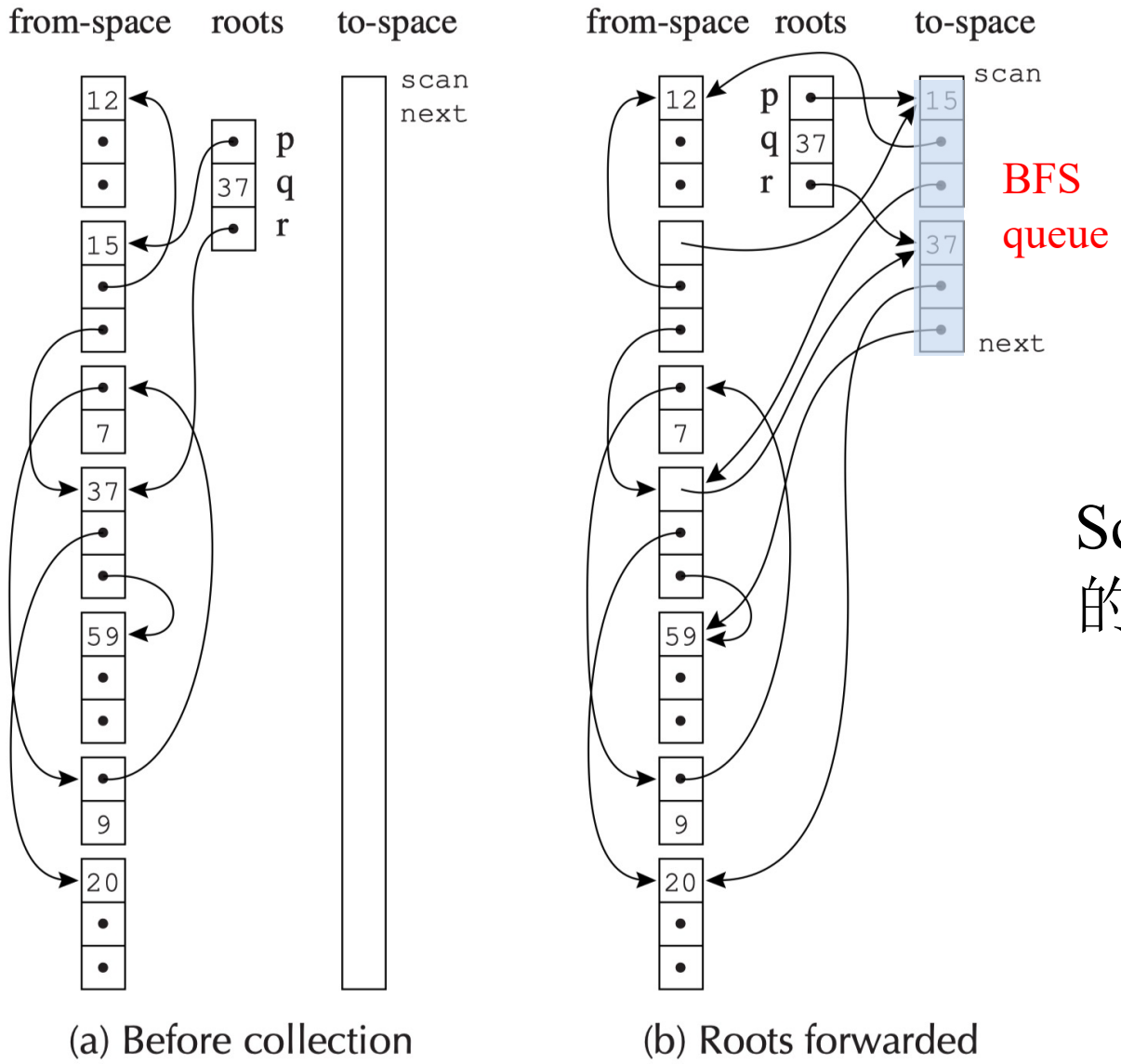
- Update the field to point to point to the to-space copy
- Move the scan pointer

Area between scan and next: the BFS queue

Example 2: Forwarding Roots



Example 2: Forwarding Roots

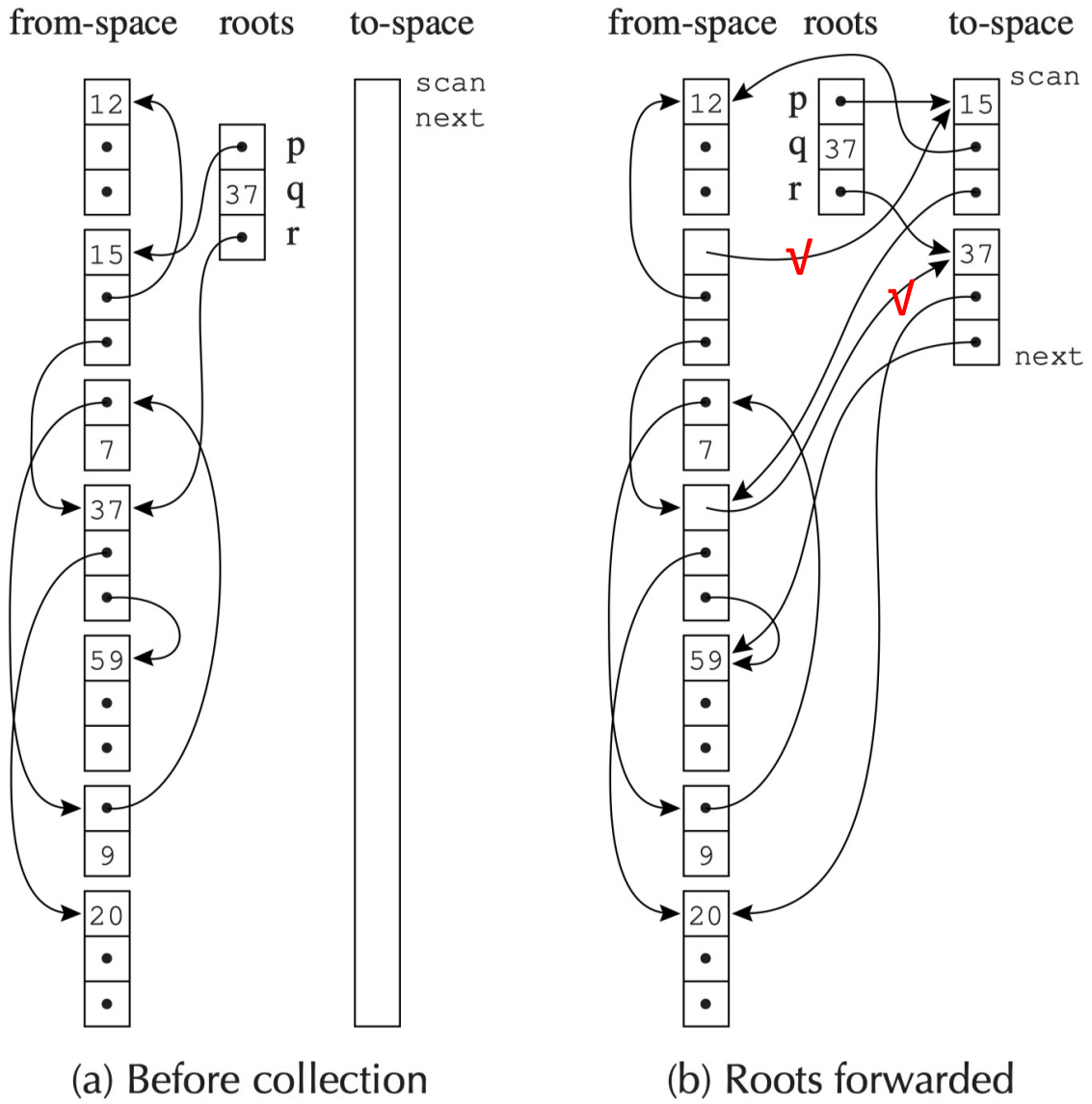


(a) Before collection

(b) Roots forwarded

Scan和next之间的是BFS的队列。

Example 2: Forwarding Roots



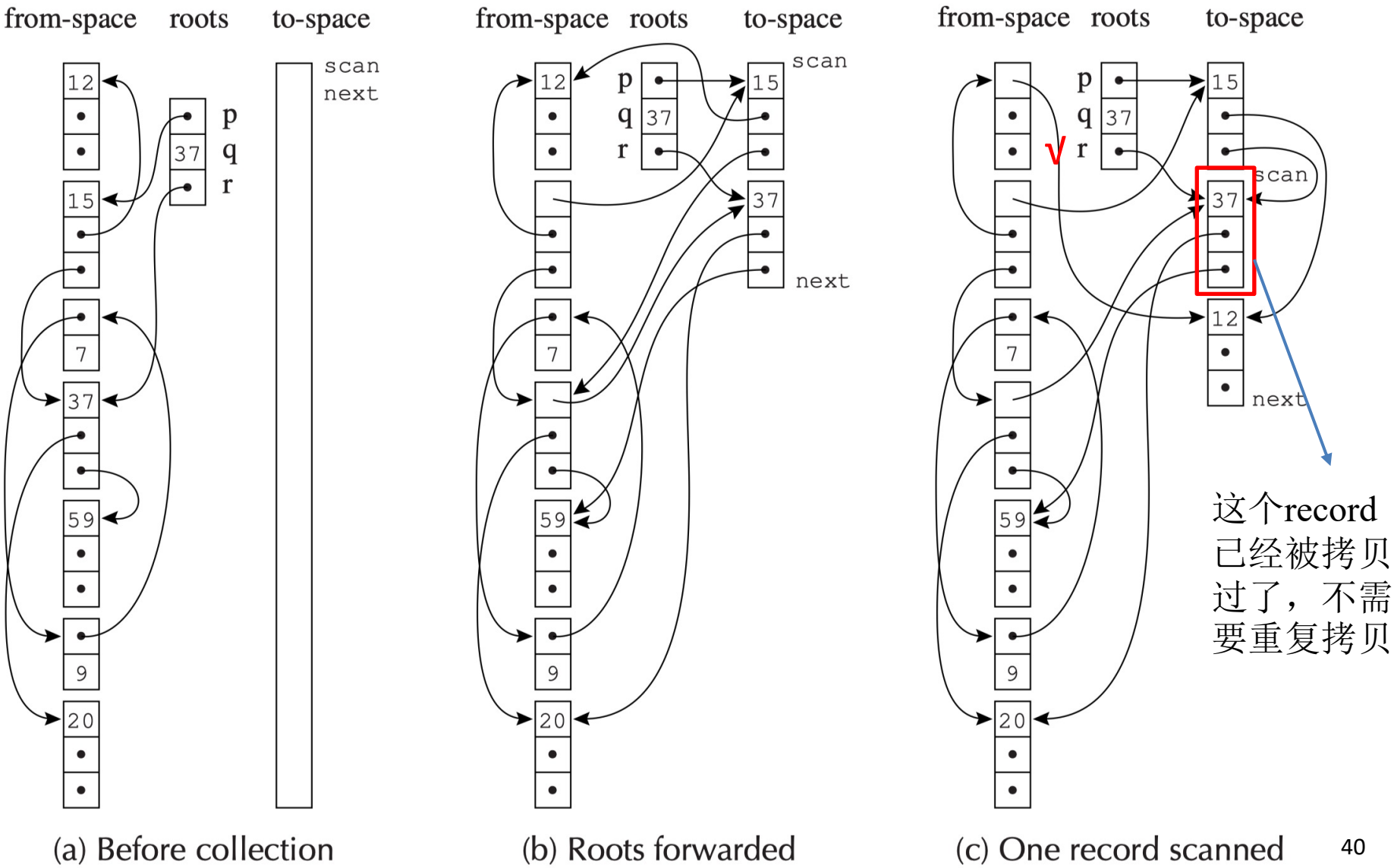
(a) Before collection

(b) Roots forwarded

Forwarding pointer

- 复用了原本from-space的records的p.f1，作为forwarding points
- 指向了to-space中records的新地址

Example 2: Forwarding More Records

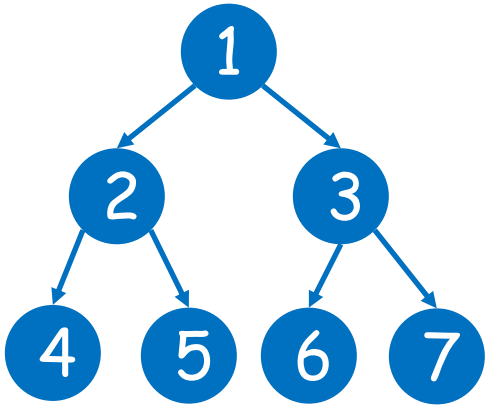


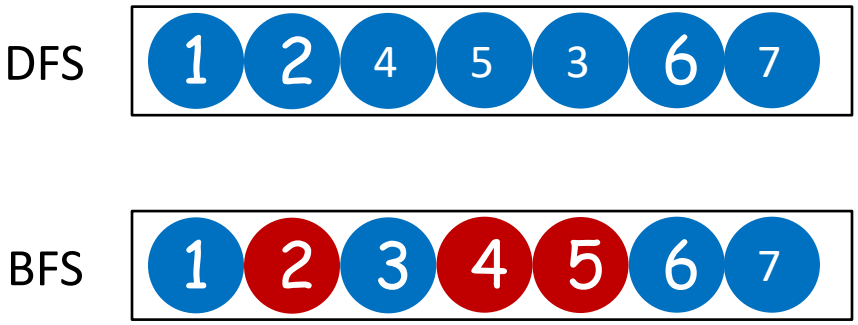
3. Copying Collection

- The Basic Idea
- Cheney's Algorithm
- Pointer Forwarding
- Locality & Improvements

Locality of Reference — BFS vs. DFS Copying

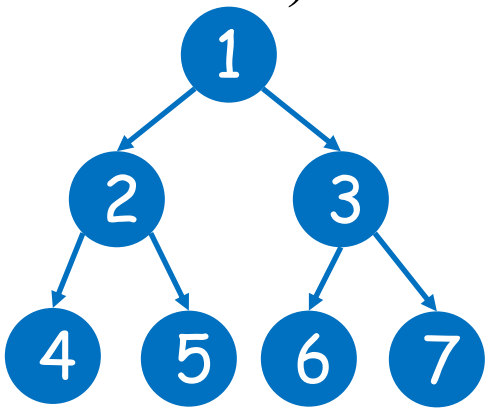
- **Locality**: after accessing address *a*, the CPU cache expects nearby addresses to be accessed soon
- Cheney's BFS is **breadth-first order**:
  - Objects at the same BFS level are stored adjacently in to-space
  - Parent and child (structurally related) end up far apart
  - Program traversal jumps widely → many cache misses

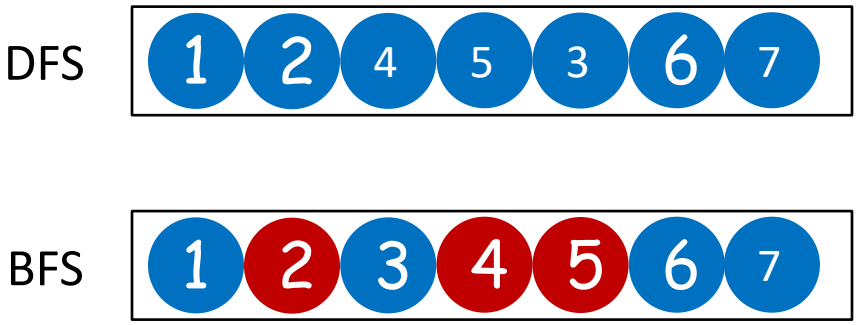




Locality of Reference — BFS vs. DFS Copying

- **Locality matters:** After the CPU fetches address *a*, the hardware pre-fetches nearby cache lines. If related objects are far apart, every pointer dereference causes a cache miss — especially costly during GC scanning.
- **DFS — Better Locality**
  - Parent is placed *next to* its children in to-space
  - Accessing a subtree → data already in cache
  - But DFS requires an explicit stack (or pointer-reversal) — extra complexity





A Hybrid Algorithm
(不考算法细节，但可能涉及locality概念)

- Partly depth-first and partly breadth-first algorithm

```
function Forward(p)
    if p points to from-space
    then if p.f1 points to to-space
            then return p.f1
            else Chase(p); return p.f1
    else return p
```

```
function Chase(p)
    repeat
        q ← next
        next ← next+ size of record p
        r ← nil
        for each field fi of record p
            q.fi ← p.fi
            if q.fi points to from-space and
q.fi.f1 does not point to to-space
            then r ← q.fi
        p.f1 ← q
        p←r
    until p = nil
```

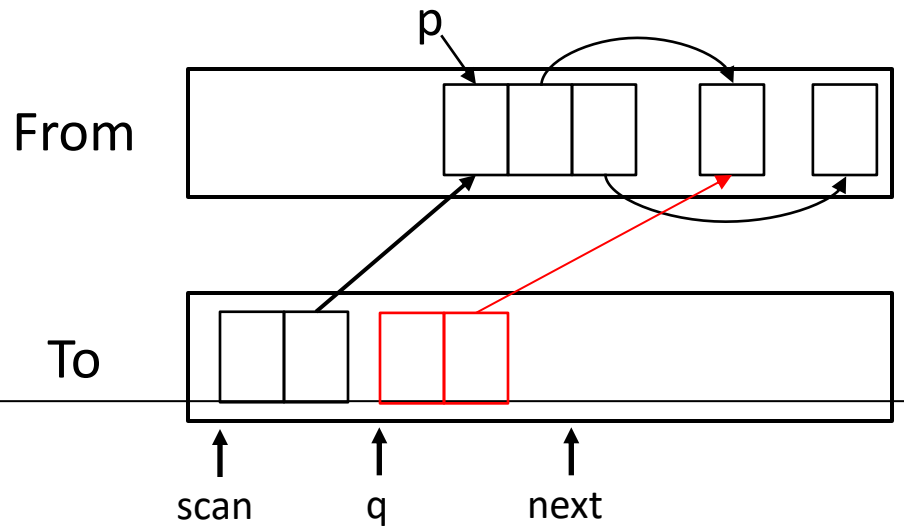
- When copying an object, try to immediately copy one of its children
- Continue depth-first as long as possible (until we hit nil or already-copied object)
- Eventually fall back to breadth-first scanning for remaining objects

Semi-depth-first Algorithm

```
function Chase(p)
    repeat
        q ← next
        next ← next + size of record p
        r ← nil
        for each field f_i of record p
            q.f_i ← p.f_i
            if q.f_i points to from-space and q.f_i.f_1 does not points to to-space then
                r ← q.f_i
        p.f_1 ← q
        p ← r
    until p = nil
```

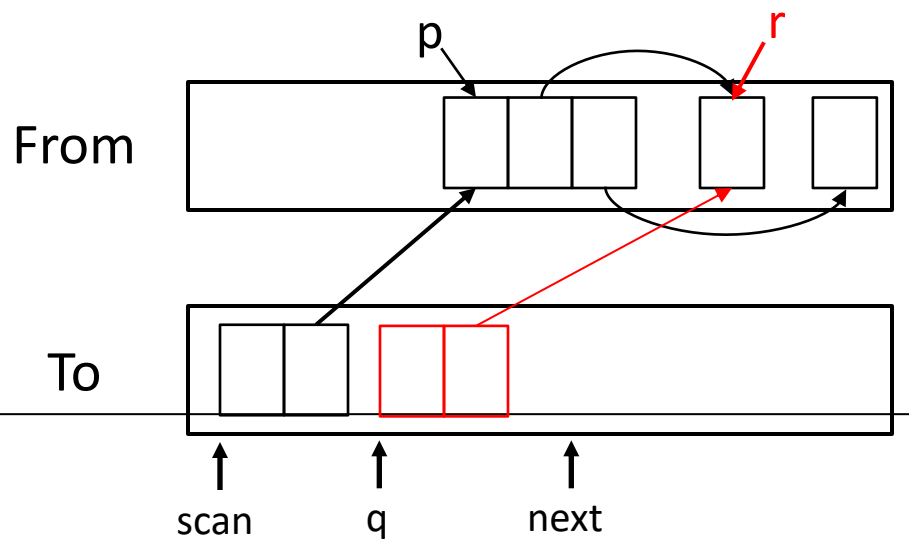
Semi-depth-first Algorithm

```
function Chase(p)
    repeat
        q ← next
        next ← next + size of record p
        r ← nil
        for each field f_i of record p
            q.f_i ← p.f_i
            if q.f_i points to from-space and q.f_i.f_1 does not points to to-space then
                r ← q.f_i
        p.f_1 ← q
        p ← r
    until p = nil
```



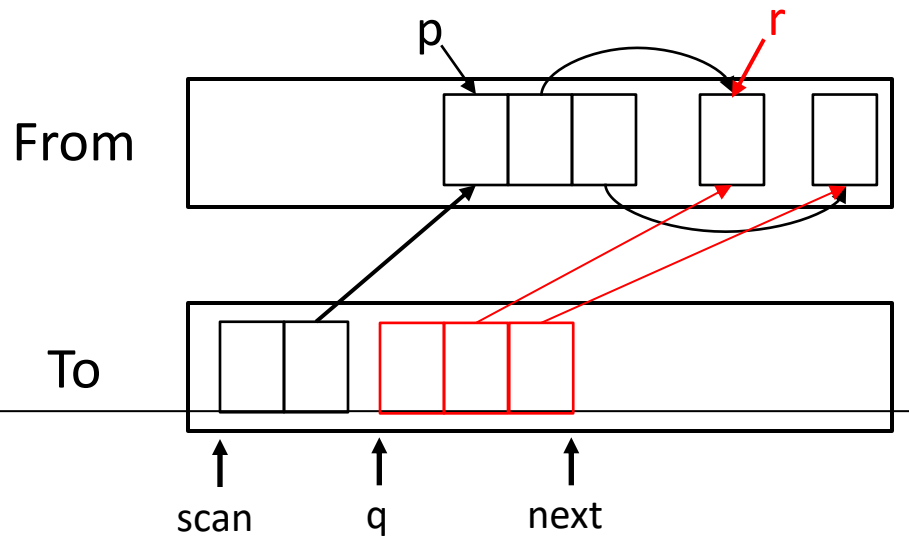
Semi-depth-first Algorithm

```
function Chase(p)
    repeat
        q ← next
        next ← next + size of record p
        r ← nil
        for each field f_i of record p
            q.f_i ← p.f_i
            if q.f_i points to from-space and q.f_i.f_1 does not points to to-space then
                r ← q.f_i
        p.f_1 ← q
        p ← r
    until p = nil
```



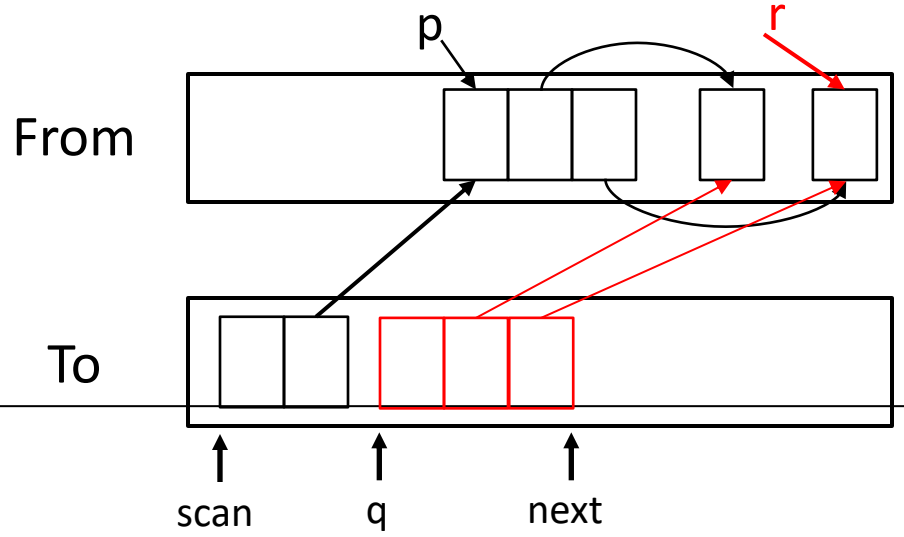
Semi-depth-first Algorithm

```
function Chase(p)
    repeat
        q ← next
        next ← next + size of record p
        r ← nil
        for each field f_i of record p
            q.f_i ← p.f_i
            if q.f_i points to from-space and q.f_i.f_1 does not points to to-space then
                r ← q.f_i
        p.f_1 ← q
        p ← r
    until p = nil
```



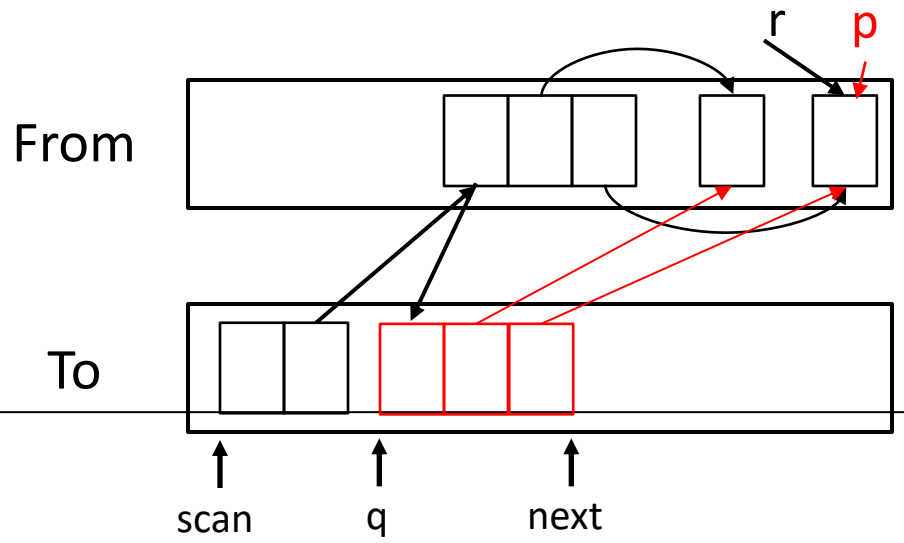
Semi-depth-first Algorithm

```
function Chase(p)
    repeat
        q ← next
        next ← next + size of record p
        r ← nil
        for each field f_i of record p
            q.f_i ← p.f_i
            if q.f_i points to from-space and q.f_i.f_1 does not points to to-space then
                r ← q.f_i
        p.f_1 ← q
        p ← r
    until p = nil
```



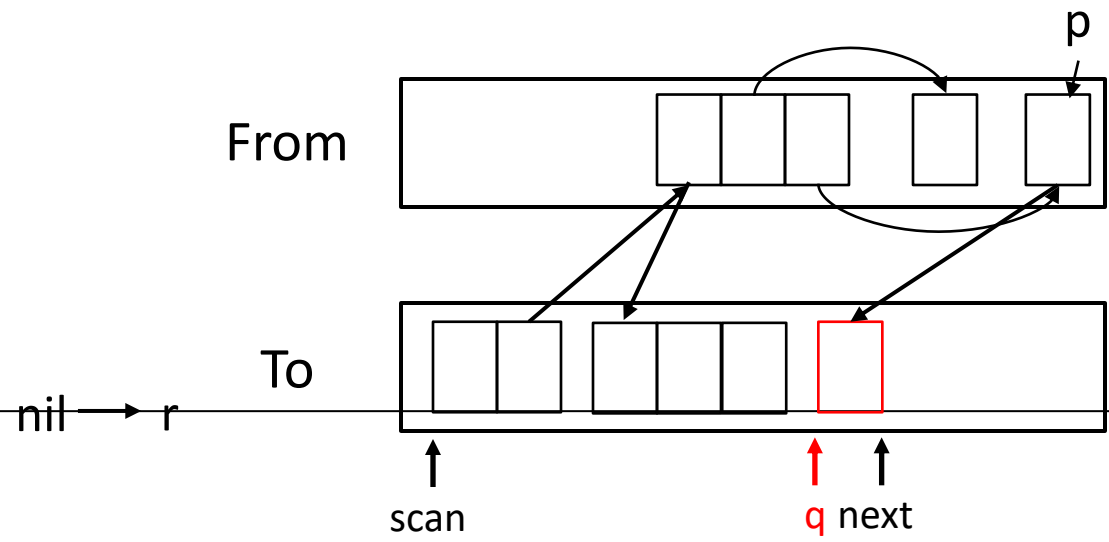
Semi-depth-first Algorithm

```
function Chase(p)
    repeat
        q ← next
        next ← next + size of record p
        r ← nil
        for each field f_i of record p
            q.f_i ← p.f_i
            if q.f_i points to from-space and q.f_i.f_1 does not points to to-space then
                r ← q.f_i
        p.f_1 ← q
        p ← r
    until p = nil
```



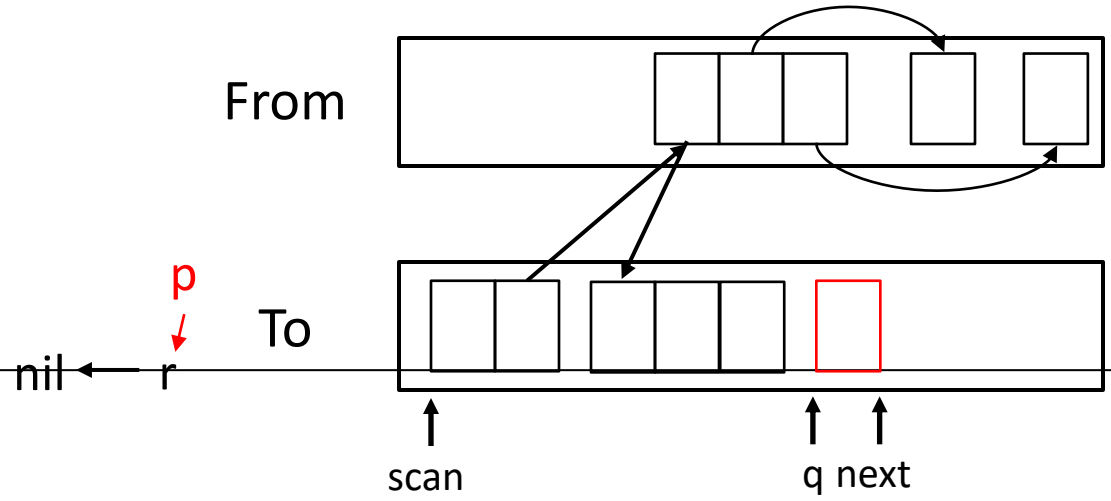
Semi-depth-first Algorithm

```
function Chase(p)
    repeat
        q ← next
        next ← next + size of record p
        r ← nil
        for each field f_i of record p
            q.f_i ← p.f_i
            if q.f_i points to from-space and q.f_i.f_1 does not points to to-space then
                r ← q.f_i
        p.f_1 ← q
        p ← r
    until p = nil
```



Semi-depth-first Algorithm

```
function Chase(p)
    repeat
        q ← next
        next ← next + size of record p
        r ← nil
        for each field f_i of record p
            q.f_i ← p.f_i
            if q.f_i points to from-space and q.f_i.f_1 does not points to to-space then
                r ← q.f_i
        p.f_1 ← q
        p ← r
    until p = nil
```



Summary: Copying Collection

- **Advantages**
  - **Very fast allocation** — just bump a pointer: p=next; next+=n
  - **No fragmentation** — live objects are compacted
  - **Simple & elegant** — no explicit free list, no stack (Cheney)
  - **Cost decided by live objects** — dead objects are never touched

- **Disadvantages**
  - **50% memory overhead** — half the heap is always idle
  - **Poor locality** — BFS order ≠ access order (mitigated by hybrid)
  - **Needs precise type info** — must distinguish pointers from integers
  - **Stop-the-world** — basic form pauses the program

5. Interface to the Compiler

- Fast Allocation
- Describing Data Layouts
- Describing Roots (Pointer Map)
- Derived Pointers

Interface to the Compiler: Overview

- Although the garbage collector is a part of "runtime", the compiler interacts with the garbage collector:
  - Generate code that **allocates** records
  - Describe **root locations** for each GC cycle
  - Describe the **layout** of heap records
  - Generate read / write **barriers** (if needed)
  - …and more

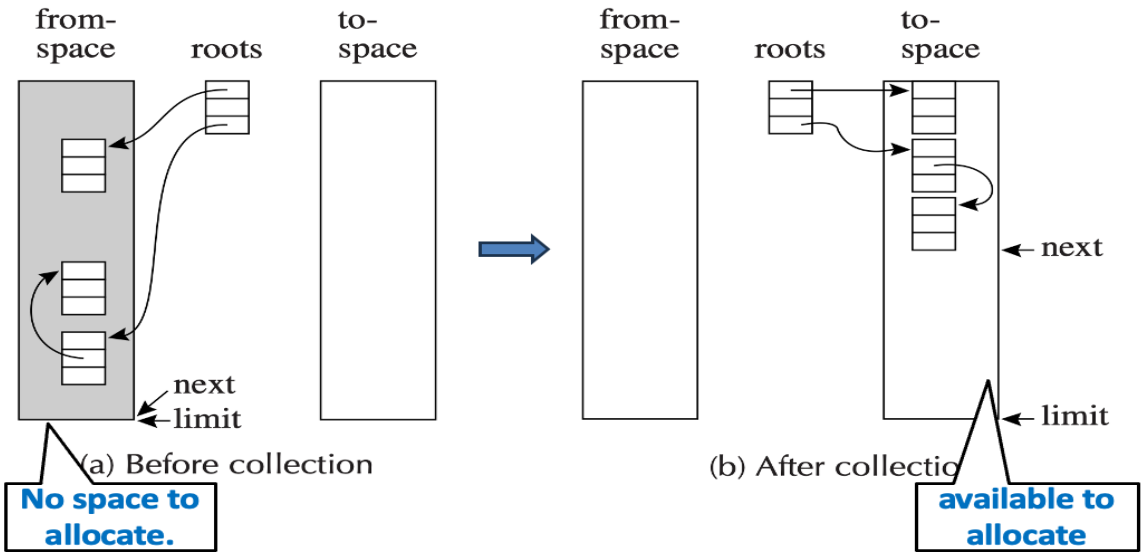
Allocation Matters for Programs

- **Functional languages** — mutation is discouraged → frequent new allocations
- **Memory-intensive apps** — objects may be accessed only once before becoming garbage

- **Empirical observation**
  - ≈ **1 in 7 instructions** is a store → at most ~1/7 word of allocation per instruction.
  - ⇒ We need allocation to cost only a handful of instructions.

Fast Allocation (for Copying Collection)

- A considerable cost is to create the heap records
- Copying collection should be used (as it is fast)
  - The allocation space is contiguous free region
  - The next free location is next and
  - The end of the region is limit



Steps in Allocation (for Copying Collection)

A straightforward implementation of allocate(N) involves these steps:

1. Call the allocate function
2. Test *next + N < limit* ? (If the test fails, call GC)
3. Move *next* to *result*
4. Clear *M[next]*, *M[next+1]*, ..., *M[next + N - 1]*
5. *next <- next + N*
6. Return from the allocate function

A. Move result into some computationally useful place

B. Store useful values into the record

(Steps A and B are not allocation overhead. — the program must perform them regardless. We will see they can replace steps 3 and 4)

Eliminate the Function Call — Inline Expansion

The steps to allocate a record of size N:

~~1. Call the allocate function~~

2. Test *next + N < limit* ? (If the test fails, call GC)
3. Move *next* to *result*
4. Clear *M[next]*, *M[next+1]*, ..., *M[next + N - 1]*
5. *next <- next + N*

~~6. Return from the allocate function~~

Inline-expand the allocate function. The compiler emits the allocation code *directly at the call site*. The slow path (space exhausted → call GC) remains a genuine function call, but this is rare.

Eliminate the Copy: Merge Step 3 with Step A

The steps to allocate a record of size N:

~~1. Call the allocate function~~

2. Test *next + N < limit* ? (If the test fails, call GC)

A. Move *next* into some computationally useful place

4. Clear *M[next]*, *M[next+1]*, ..., *M[next + N - 1]*

5. next <- next + N

~~6. Return from the allocate function~~

~~A. Move result into some computationally useful place~~

B. Store useful values into the record

Solution: The compiler arranges for **next** to land in the register that will hold the new object pointer — no intermediate *result* variable is needed.

Eliminate the Zero-Fill: Subsume Step 4 into Step B

The steps to allocate a record of size N:

~~1. Call the allocate function~~

2. Test *next + N < limit* ? (If the test fails, call GC)

A. Move *next* into some computationally useful place

~~4. Clear *M[next], M[next+1], ..., M[next + N - 1]* (Subsumed by Step B)~~

5. next <- next + N

~~6. Return from the allocate function~~

~~A. Move result into some computationally useful place~~

B. Store useful values into the record

Solution: If Step B provably covers *all* fields before any GC point can observe them, the zero-fill is redundant and can be dropped.

Share Steps 2 &5 Across Multiple Allocations

The steps to allocate a record of size N:

~~1. Call the allocate function~~

2. Test *next + N < limit* ? (If the test fails, call GC)

A. Move *next* into some computationally useful place

~~4. Clear *M[next], M[next+1], …, M[next + N - 1]*~~

5. next <- next + N

~~6. Return from the allocate function~~

~~A. Move result into some computationally useful place~~

B. Store useful values into the record

Step 2 and 5 cannot be eliminated. But they can be shared among multiple allocations.

Share Steps 2 &5 Across Multiple Allocations

Individual allocation (2 objects, sizes N_1 and N_2):

```
// Naïve: two separate checks
if next+N₁ ≥ limit: call gc result₁ ← next; next ← next+N₁
if next+N₂ ≥ limit: call gc result₂ ← next; next ← next+N₂
```

Batched (shared check):

```
// One combined check
if next+N₁+N₂ ≥ limit:
   call gc
   result₁ ← next result₂ ← next + N₁ next ← next + N₁ + N₂
```

Key insight: If the compiler can identify several allocations that will all occur before the next potential GC point, it can batch them — performing a single bounds check and a single pointer advance for the whole group.

Fast Allocation (for Copying Collection)

- Step 2: Test *next* + *N* < *limit* ? (If the test fails, call GC)
- Step 5: *next* <- *next* + *N*

- By keeping next and limit in registers, steps 2 and 5 can be done in a total of three instructions

- By this combination of techniques, the cost of allocating a record – and then eventually garbage collecting it - can be brought down to about four instructions

5. Interface to the Compiler

- Fast Allocation
- Describing Data Layouts
- Describing Roots (Pointer Map)
- Derived Pointers

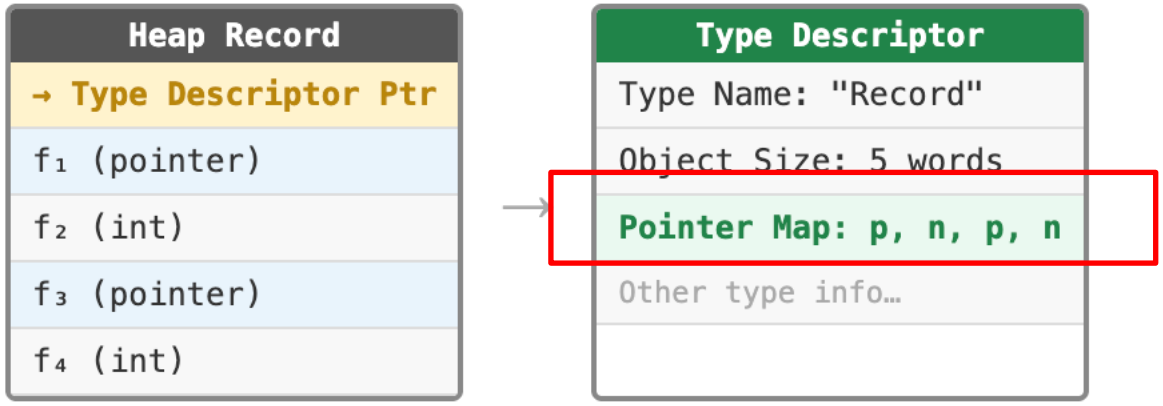
Describing Data Layouts: Motivation

```
scan ← next ← beginning of to-space
for each root r
    r ← Forward(r)
while scan < next
    for each field f_i at scan
        scan.f_i ← Forward(scan.f_i)          // which fields are pointers?
    scan ← scan + size of record at scan      // record size?
```

- **Two questions per record**
  1. **Size** — how far to advance scan?
  2. **Field types** — which fields are pointers (to call Forward)?

Type Descriptors

- Solution: Each heap record holds a pointer to a type descriptor stored once per type in the data segment.



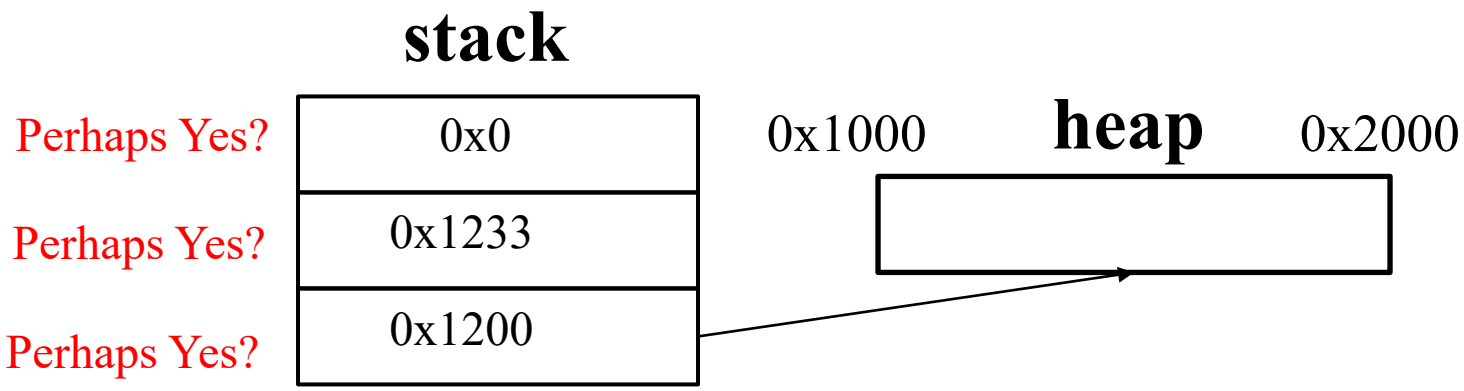
- Overhead
  - For OO languages: no extra overhead
    - objects need type descriptors (introduced later)
  - For statically typed language: one-word overhead

5. Interface to the Compiler

- Fast Allocation
- Describing Data Layouts
- Describing Roots (Pointer Map)
- Derived Pointers

The Root Identification Problem

- The GC must trace from **all** roots. Roots live in:
  - Machine **registers**
  - **Local variables** in stack frames
  - **Temporary** variables
  - **Global** variables



Conservative GC treats anything that "looks like" a pointer as one — but this is imprecise and prevents object movement.

Pointer Map (Stack Map)

- **Idea:** The compiler knows the type of every variable at compile time → generate a map at each *GC-safe point*.

- Key properties
  - The compiler knows all types at compile time
  - Each temp / stack slot is marked **pointer** or **non-pointer**
  - All maps generated statically — zero runtime cost to build
  - GC uses the maps to identify roots precisely — no guessing

- **Placement constraint:**
  - Live sets change at every program point. A pointer map per instruction would be enormous — impractical.

Pointer Map Placement

- Pointer maps are only needed at **GC points** — places where collection can actually be triggered:

GC Point 1: Allocation sites
GC fires when next + N ≥ limit. Insert a pointer map immediately before each alloc_record call.

GC Point 2: All function calls
Any call may *indirectly* trigger allocation → GC deep in the call chain. Insert a pointer map before every call.

Propagating Pointer Info Through All Compiler Phases

Compiler Phase	Action Required for GC Support
Type Checking (AST)	Full types known for all expressions — this is the authoritative source of pointer information
IR / Temp Generation	Tag each temp variable as POINTER or NON_POINTER in a temp_map
Stack Spilling	When a temp is spilled, mark the spill slot in the activation frame as pointer or non-pointer
Register Allocation	Transfer pointer tags from temps → the physical registers they are assigned to
Code Emission	At each GC-safe point, emit a pointer map entry: {return address → list of pointer registers/slots}

The "is-pointer" annotation is a first-class attribute that flows unchanged through every compilation phase. **Efficient, precise GC requires compiler cooperation from start to finish.**

5. Interface to the Compiler

- Fast Allocation
- Describing Data Layouts
- Describing Roots
- Derived Pointers

What Is a Derived Pointer

- A pointer computed from a **base pointer** that doesn't point to the start of a heap object.
- **Example: a[i - 2000]**

```
// Source
x = a[i - 2000]

// Compiled
t1 ← a - 2000       // derived pointer! (may point before the array)
t2 ← t1 + i
t3 ← M[t2]
```

- a is the **base pointer** (points to start of the array object)
- t1 is a **derived pointer** (points 2000 bytes *before* the object)
- The GC **cannot** use t1 as a root — it doesn't point to a valid object header

What is a Derived Pointer

- A pointer computed from a **base pointer** that doesn't point to the start of a heap object.

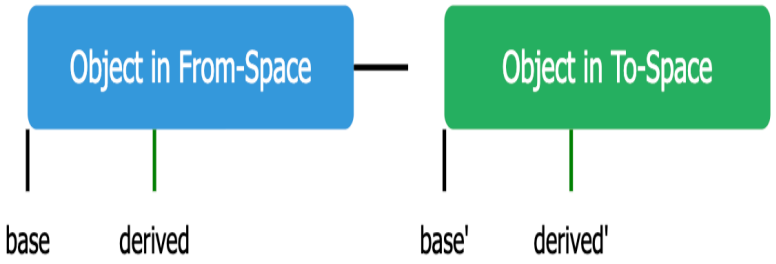
- Three variants
  - Pointer to the **interior** of an object (e.g., element address inside an array)
  - Pointer **before** the start (as in the example above)
  - Pointer **past the end** (e.g., past-the-end sentinel)

Why are Derived Pointers Problematic?

- In copying collection, when an object moves from from-space to to-space, **every pointer to it must be updated**. The update rule differs for base vs. derived pointers:

Base pointer update (straightforward):

```
base' ← Forward(base) // Forward finds the new location of the object via forwarding pointer
```



Derived pointer update (cannot use Forward):

```
// Forward(derived) is WRONG — derived doesn't point to an object!
// Correct update:
derived' ← derived + (base' − base)
```

derived' = derived + (base' - base)

The offset between base and derived is *preserved* by the move. The GC must know which base to use.

Solutions for Derived Pointers

- **1. Annotate in pointer map:** record "t1 is derived from base a"; GC forwards a first, then adjusts t1 by the same delta.

- **2. Keep base pointer live:** a derived pointer extends the liveness of its base; register allocator must keep a alive alongside t1 — may cost an extra register.

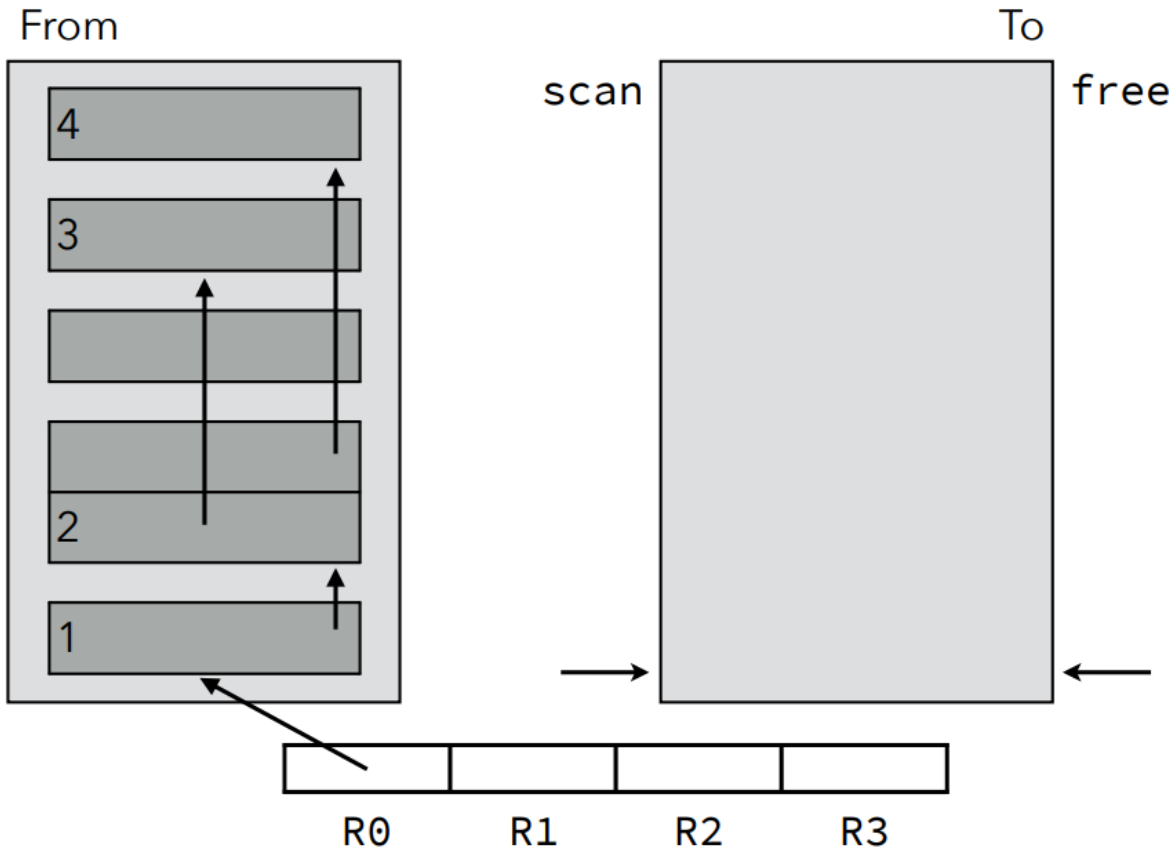
- **3. Avoid derived pointers:** rewrite code to compute M[a + (i − 2000)] without materialising the interior address.

Summary

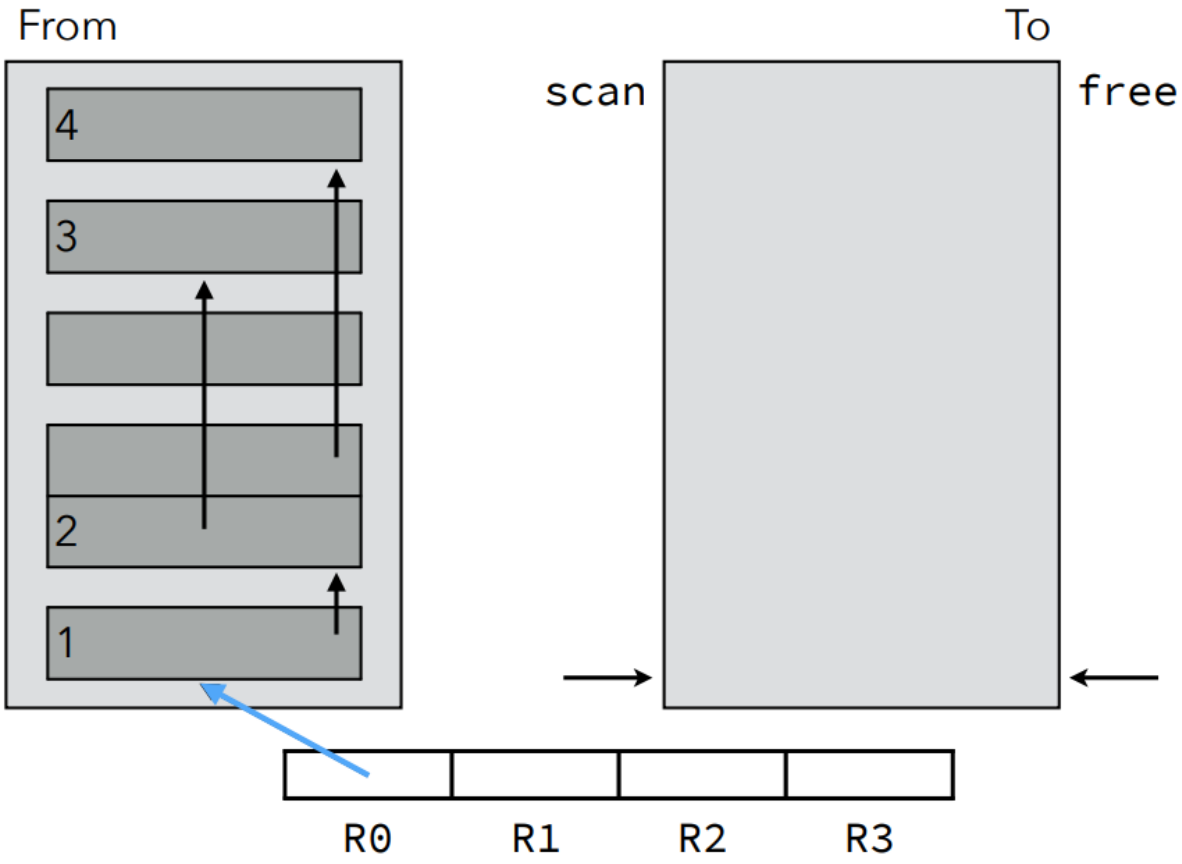
- Mark-and-sweep collection
- Reference counting
- Copying collection
- Generational Collection (书上有，但不考)
- Incremental Collection (书上有，但不考)
- Interface to the Compiler

Thank you all for your attention

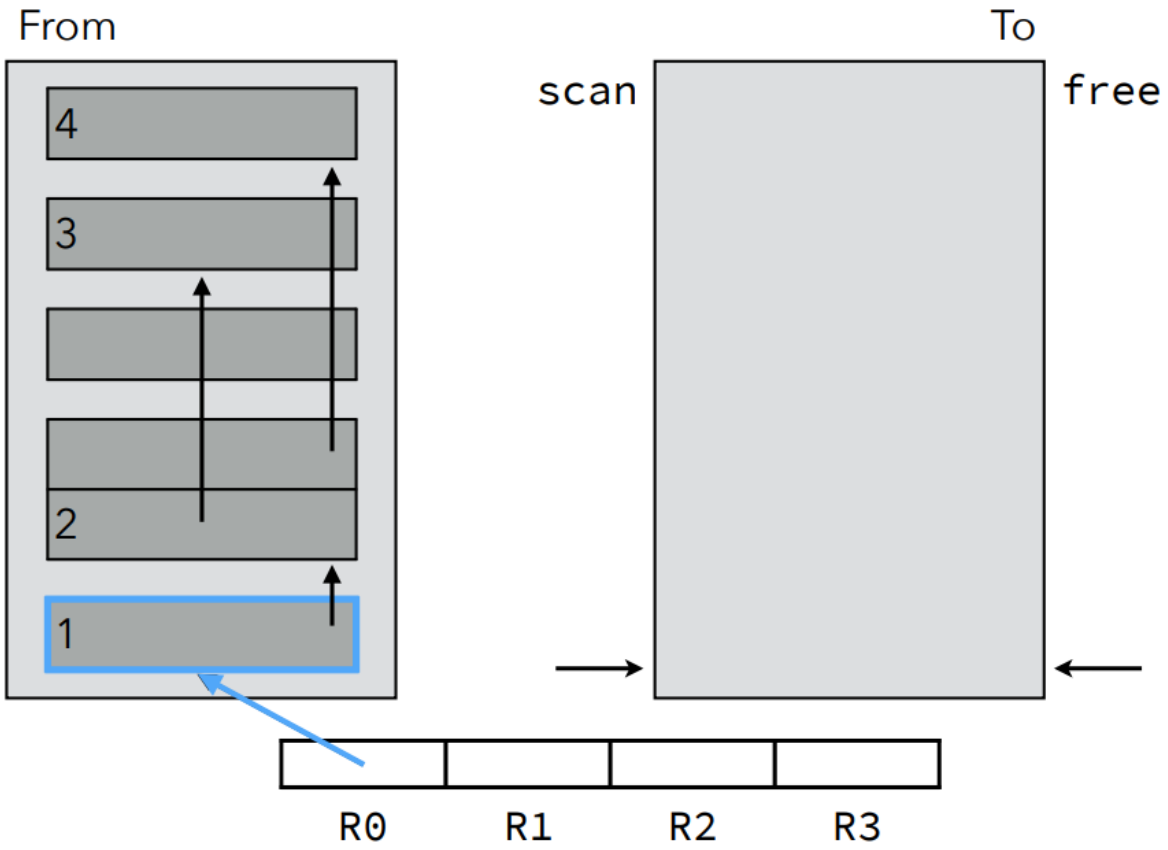
Example: Cheney's Copying GC



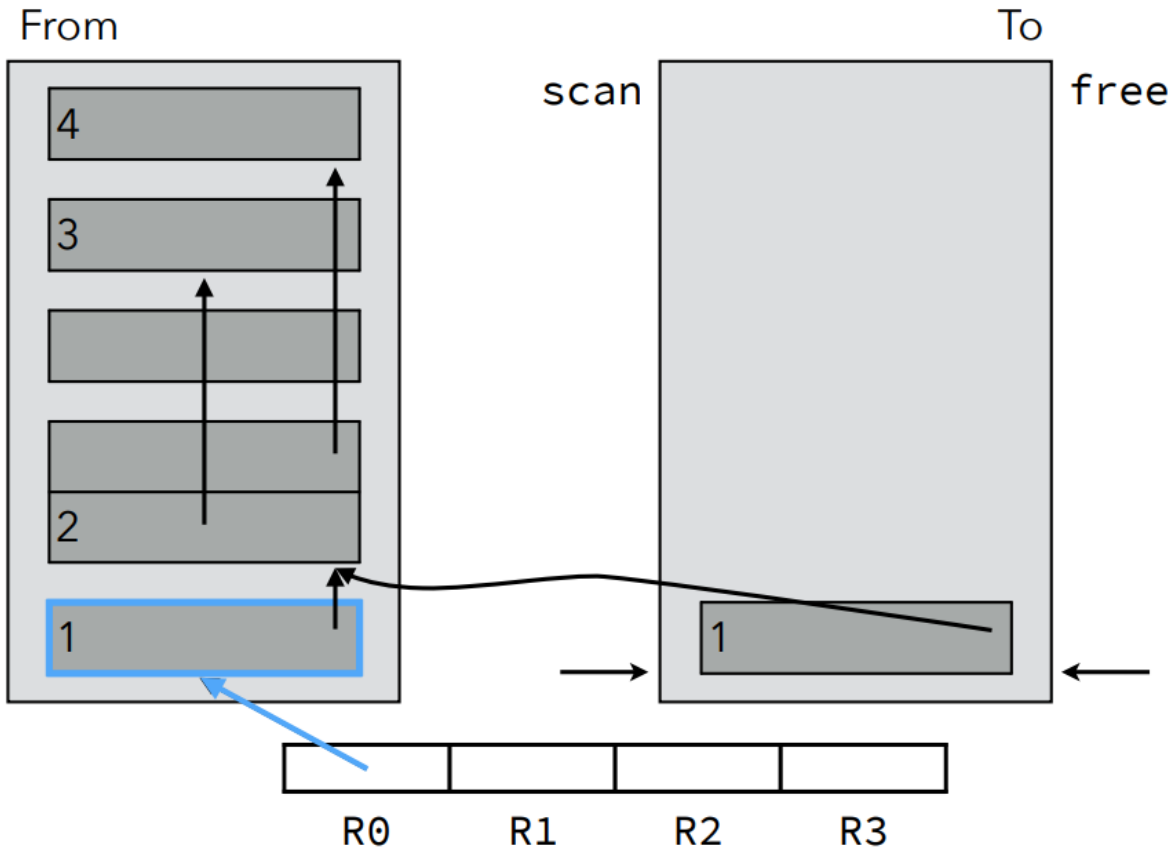
Example: Cheney's Copying GC



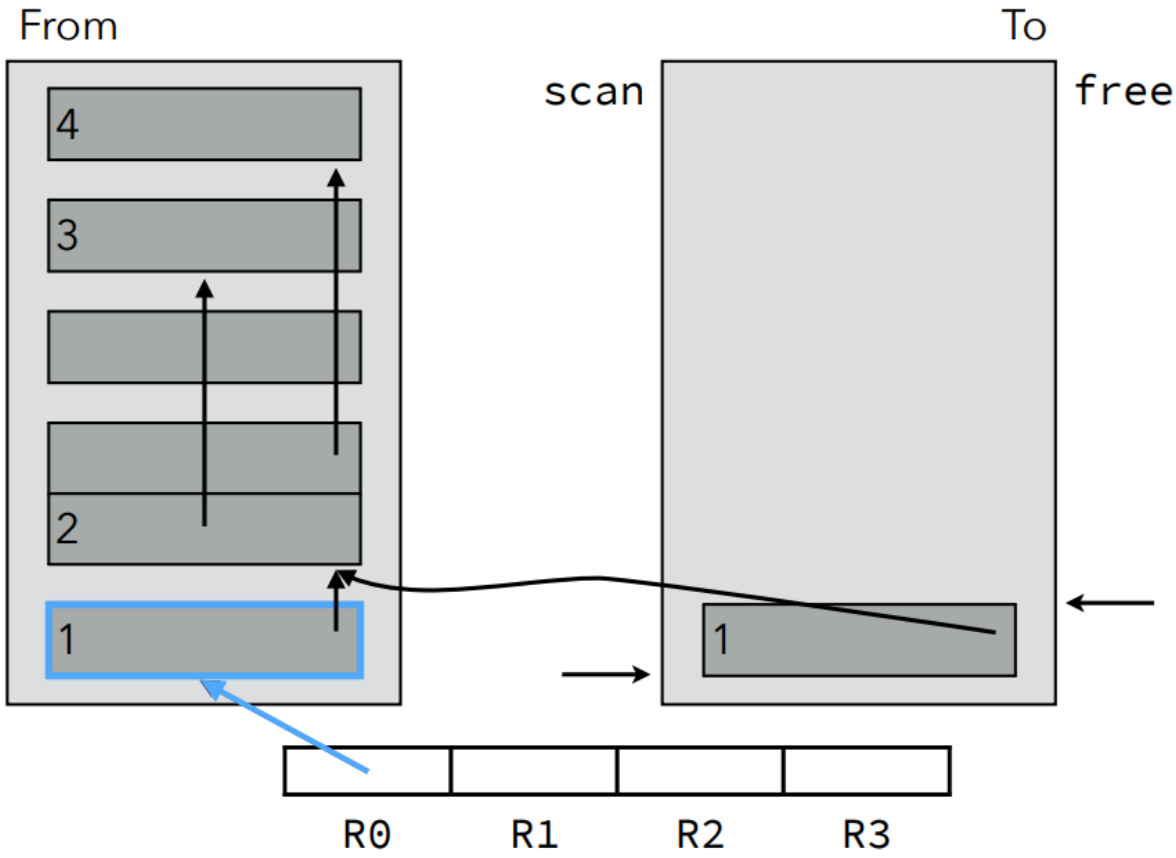
Example: Cheney's Copying GC



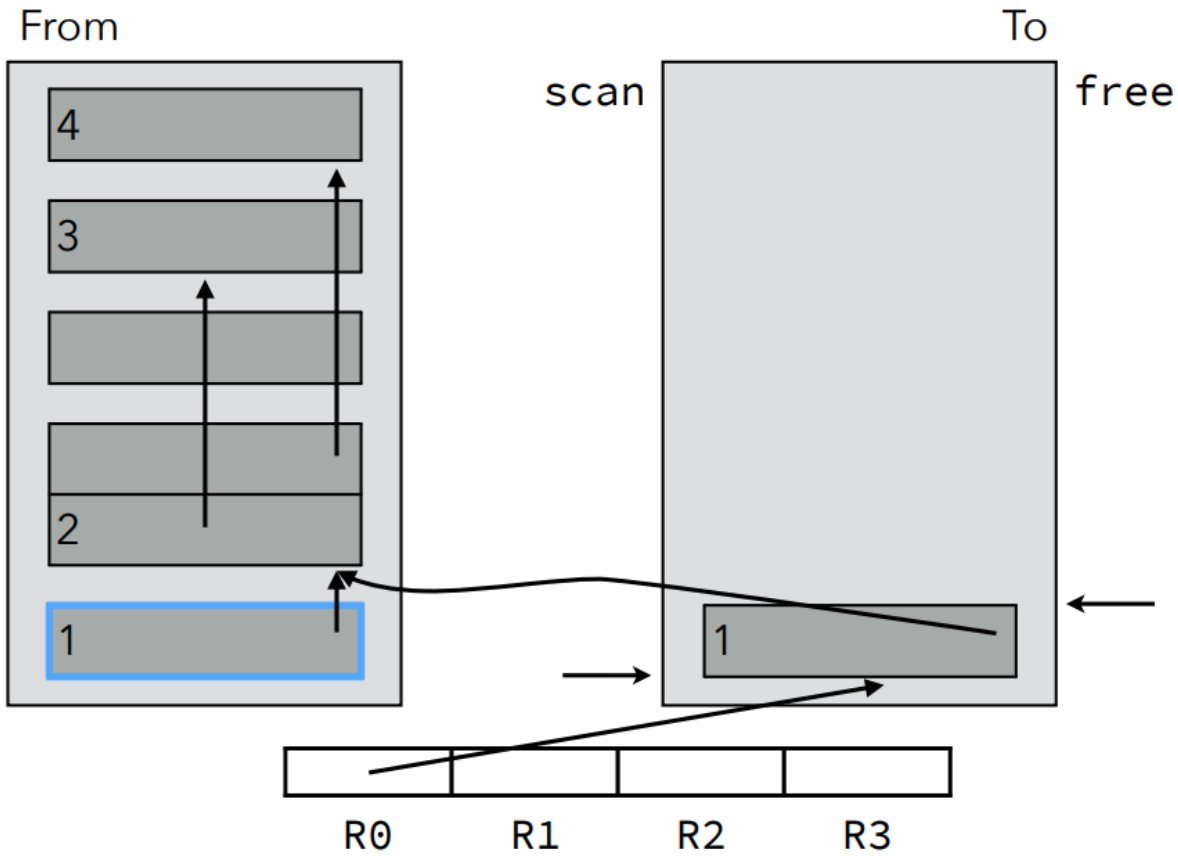
Example: Cheney's Copying GC



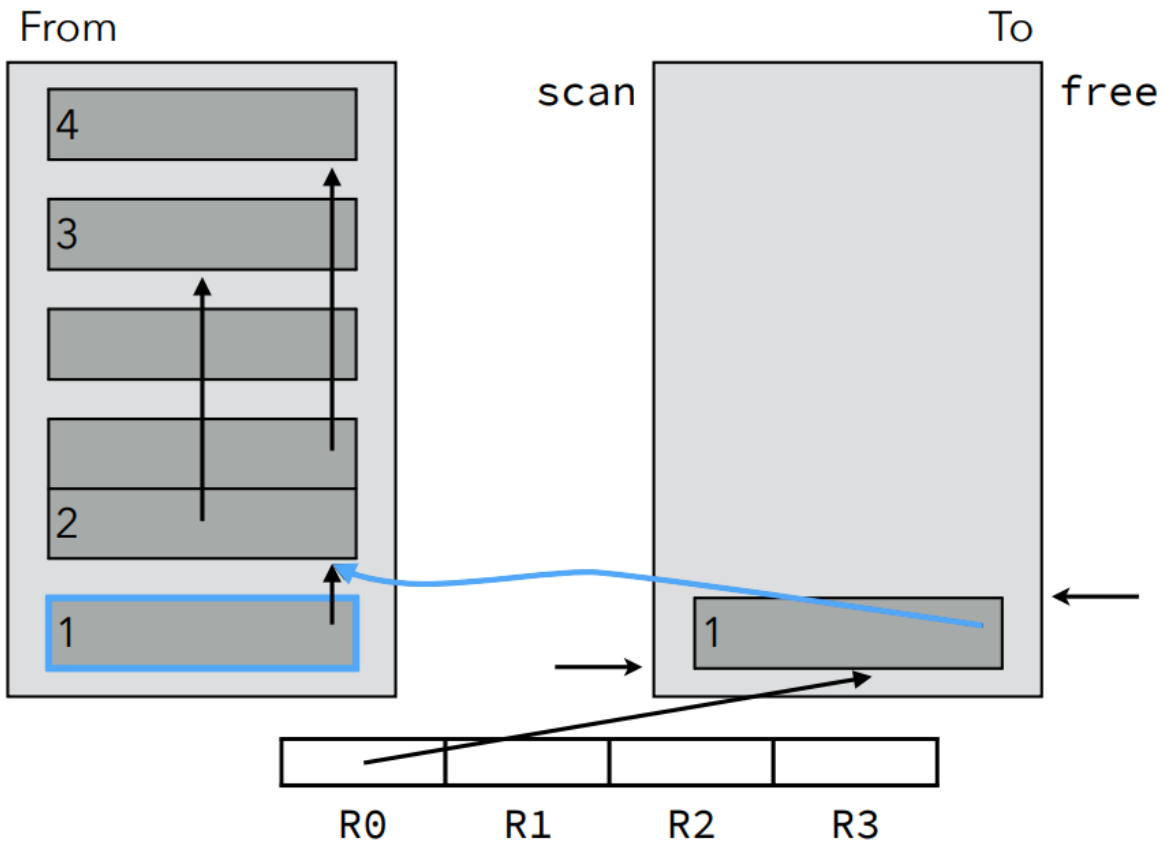
Example: Cheney's Copying GC



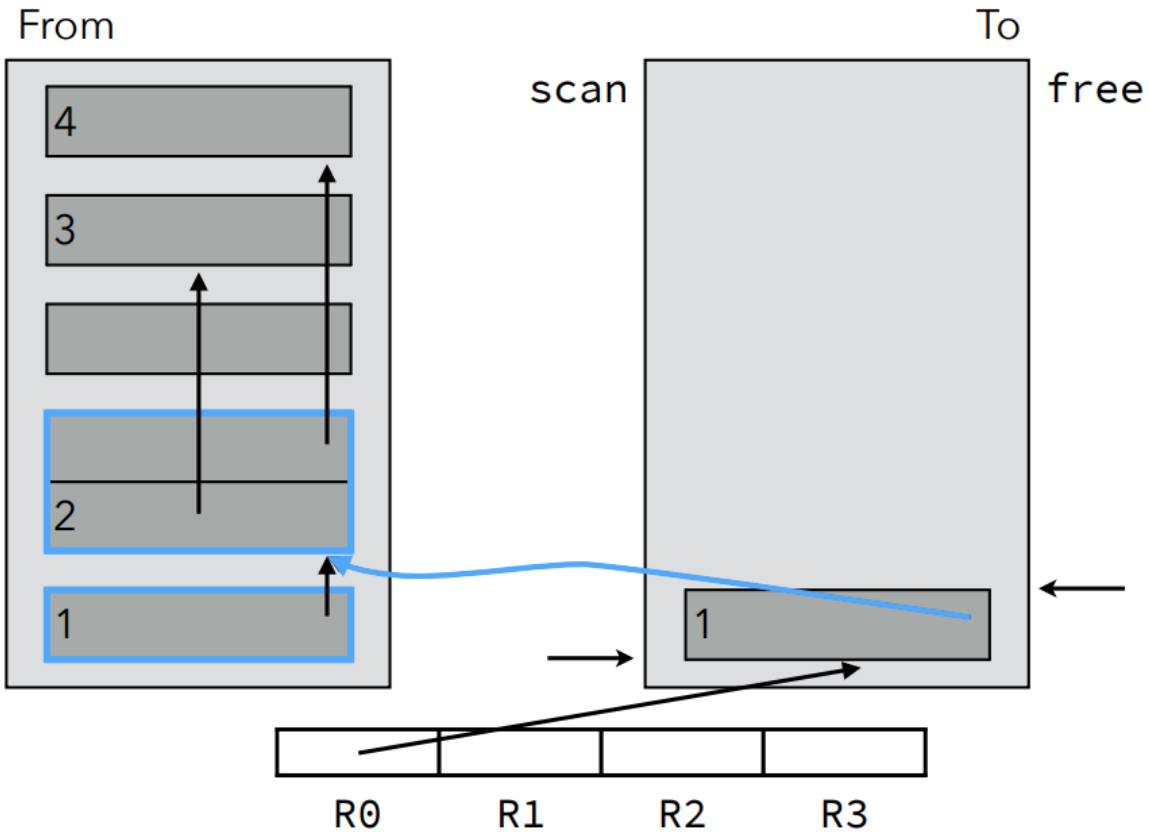
Example: Cheney's Copying GC



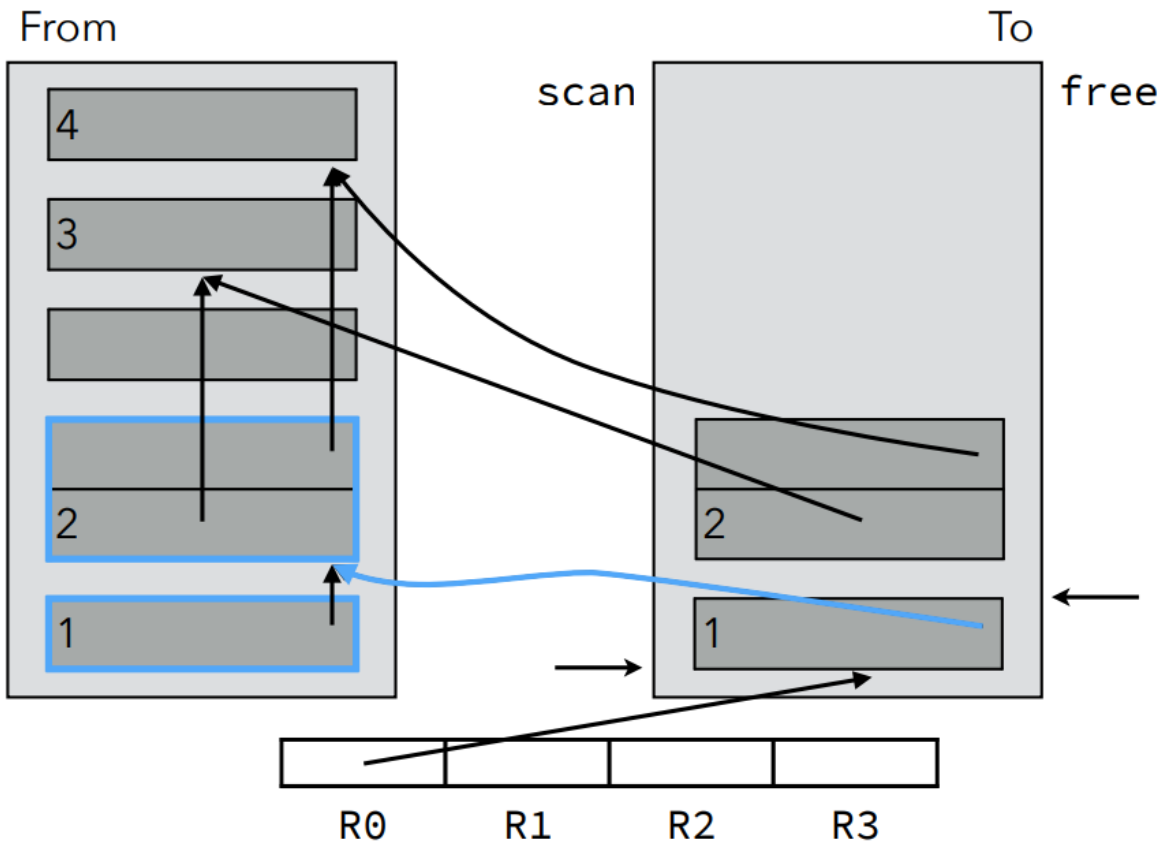
Example: Cheney's Copying GC



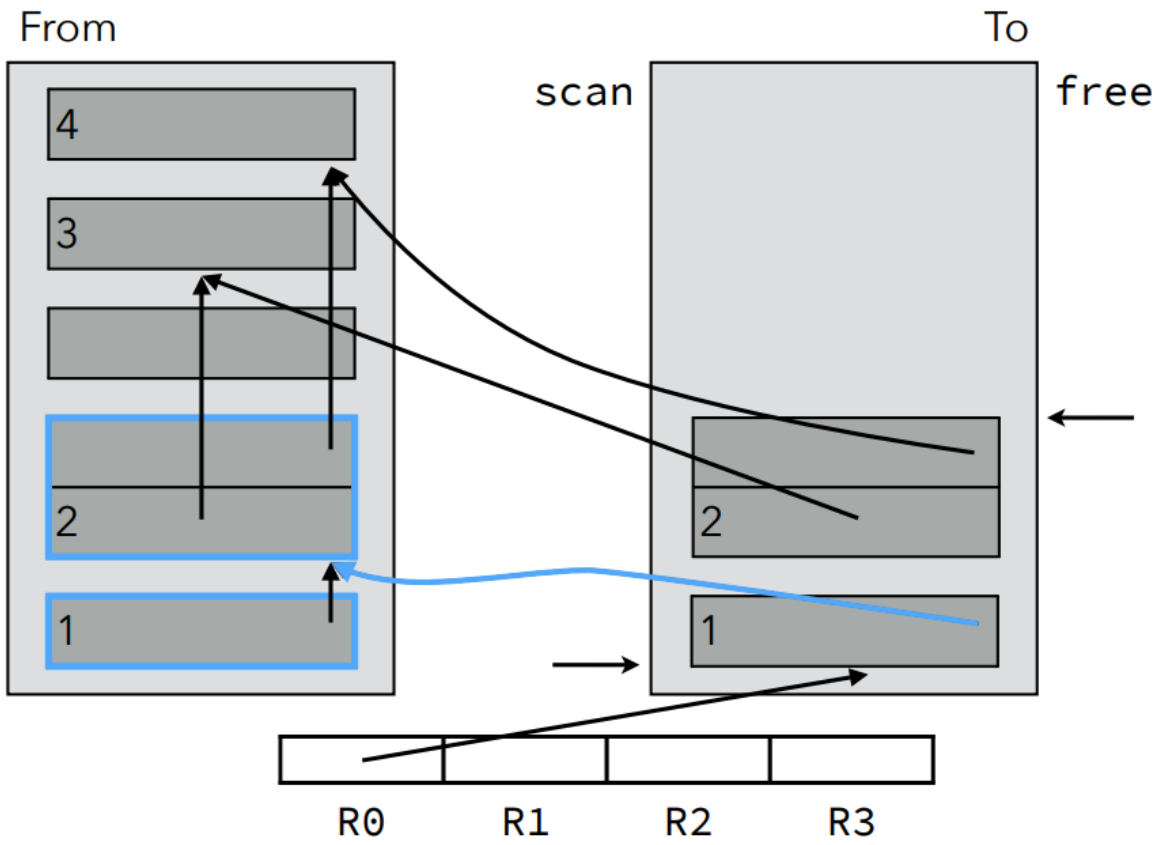
Example: Cheney's Copying GC



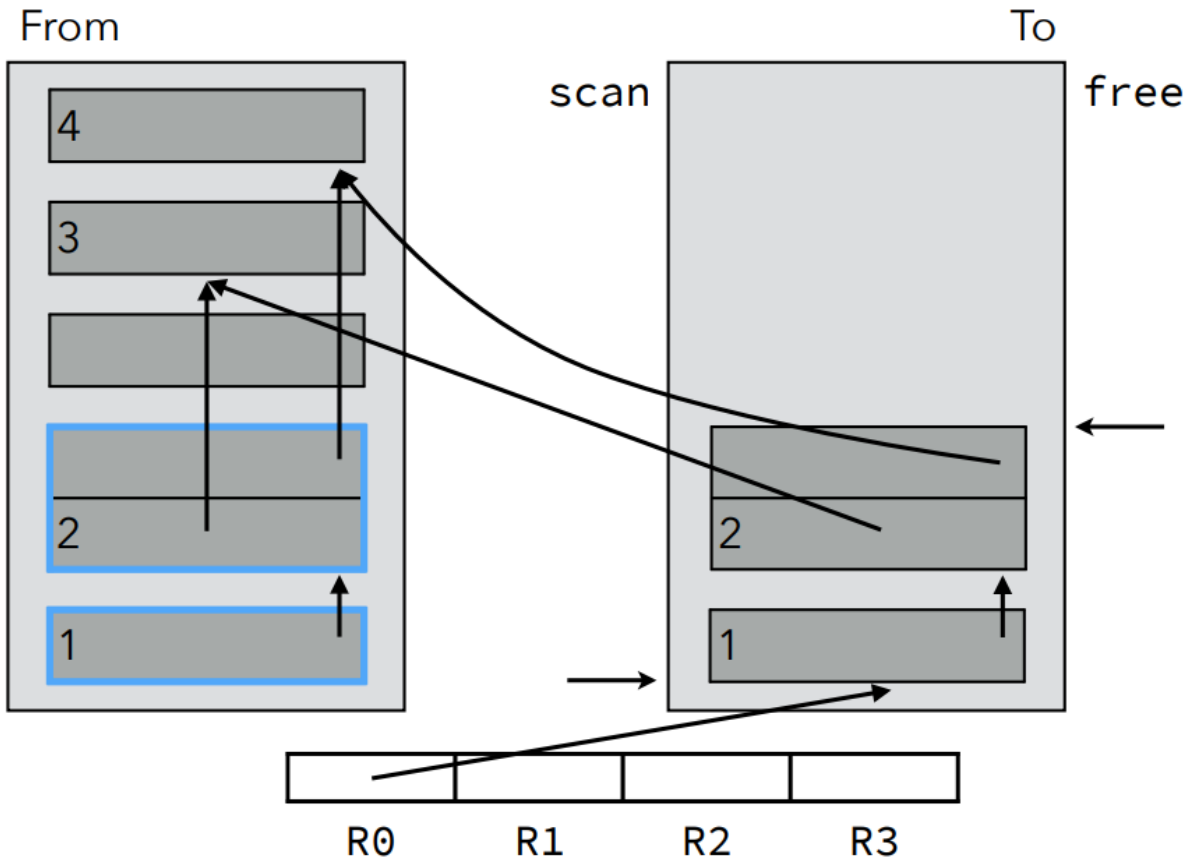
Example: Cheney's Copying GC



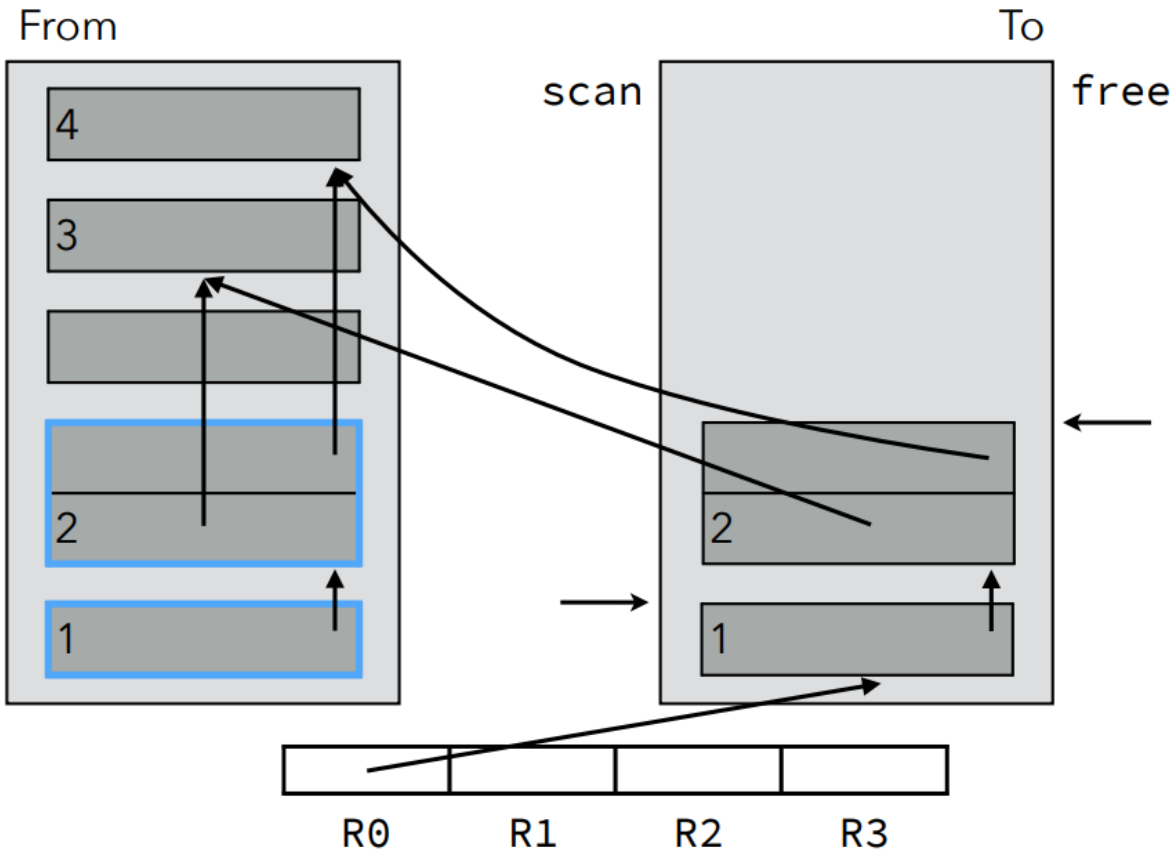
Example: Cheney's Copying GC



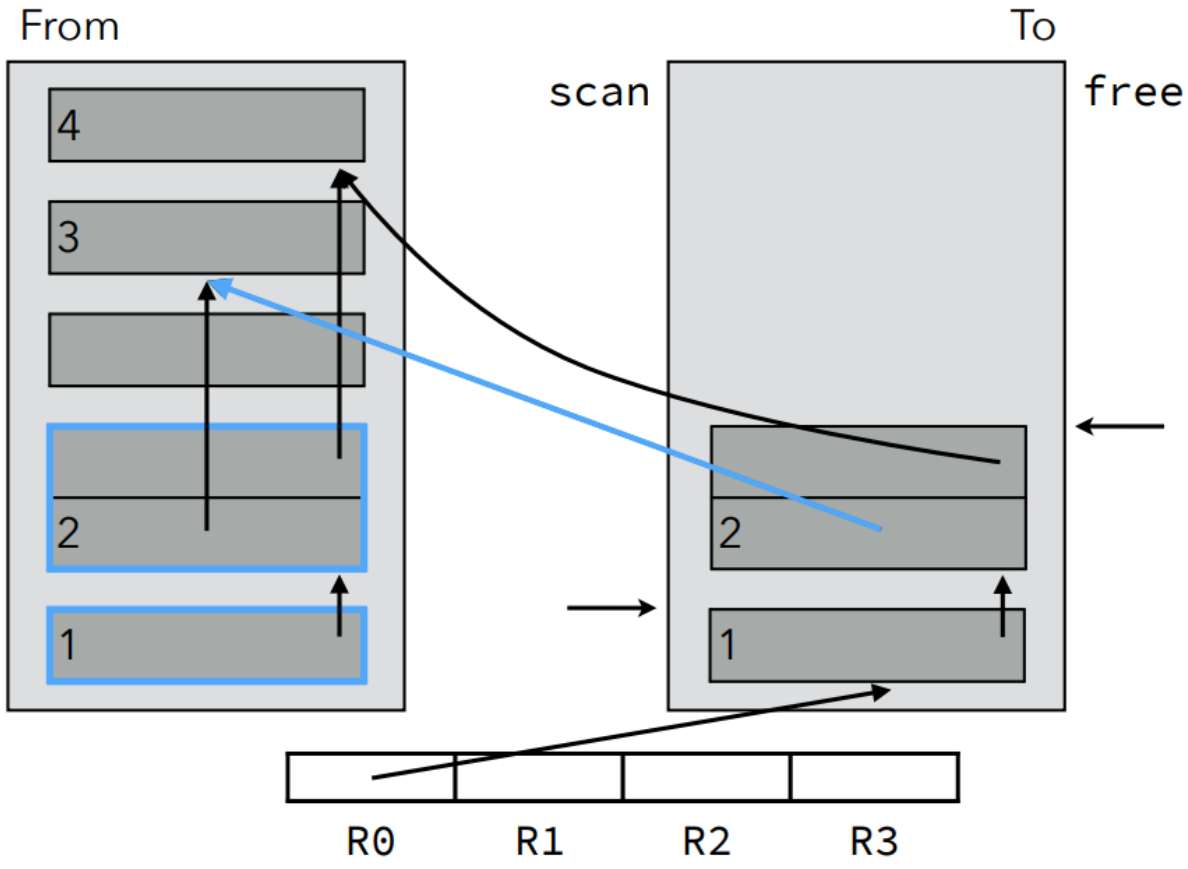
Example: Cheney's Copying GC



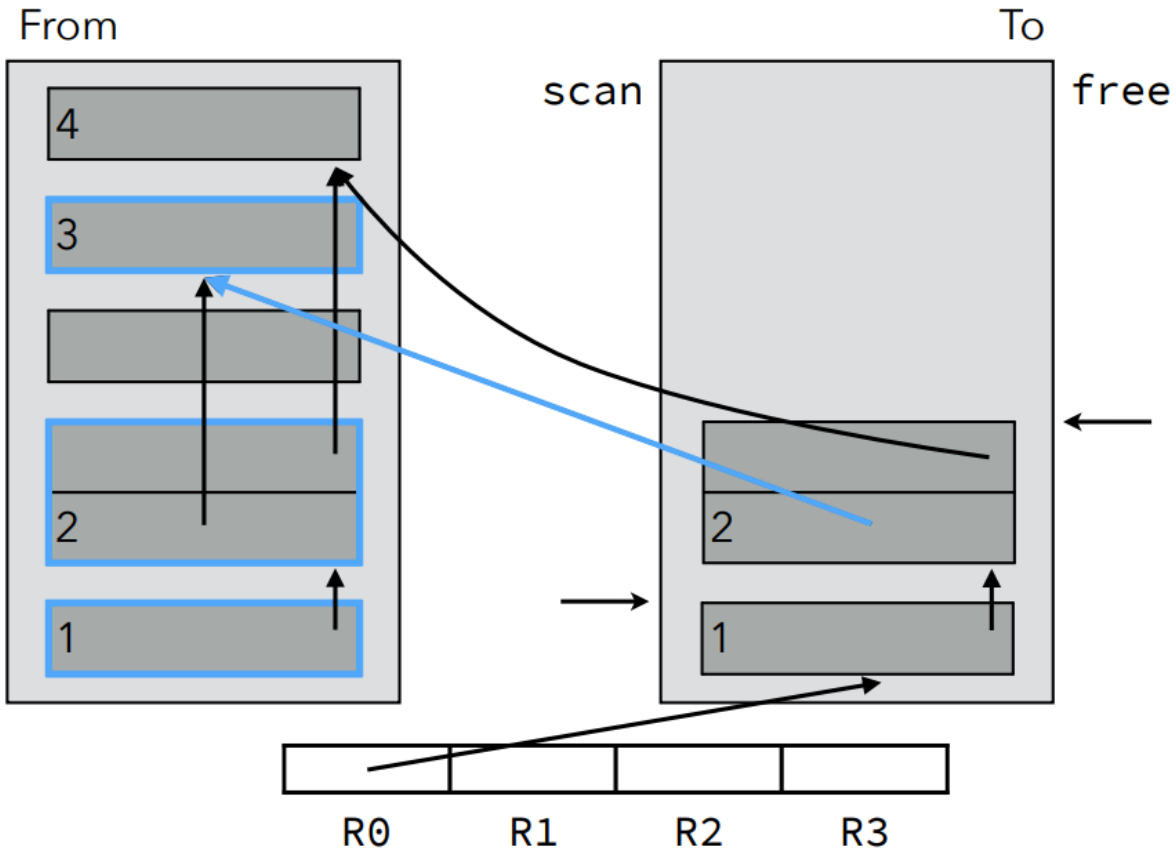
Example: Cheney's Copying GC



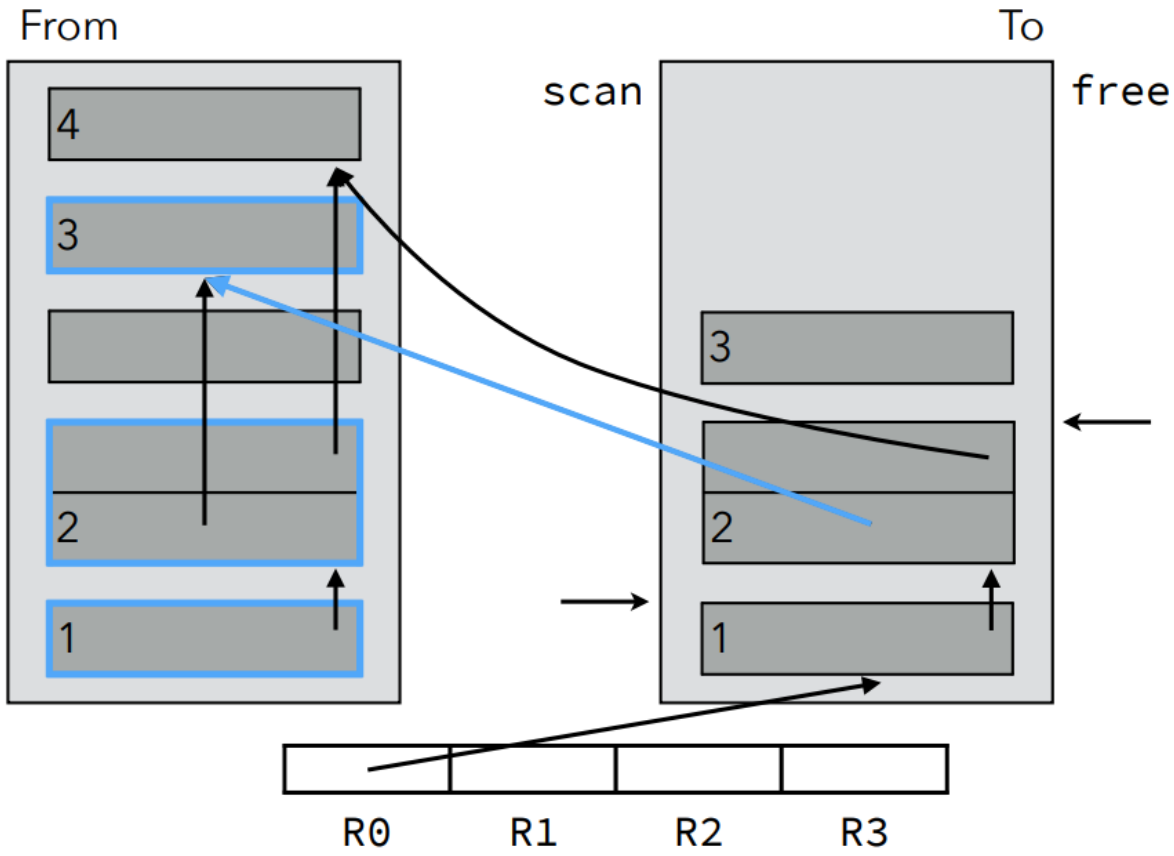
Example: Cheney's Copying GC



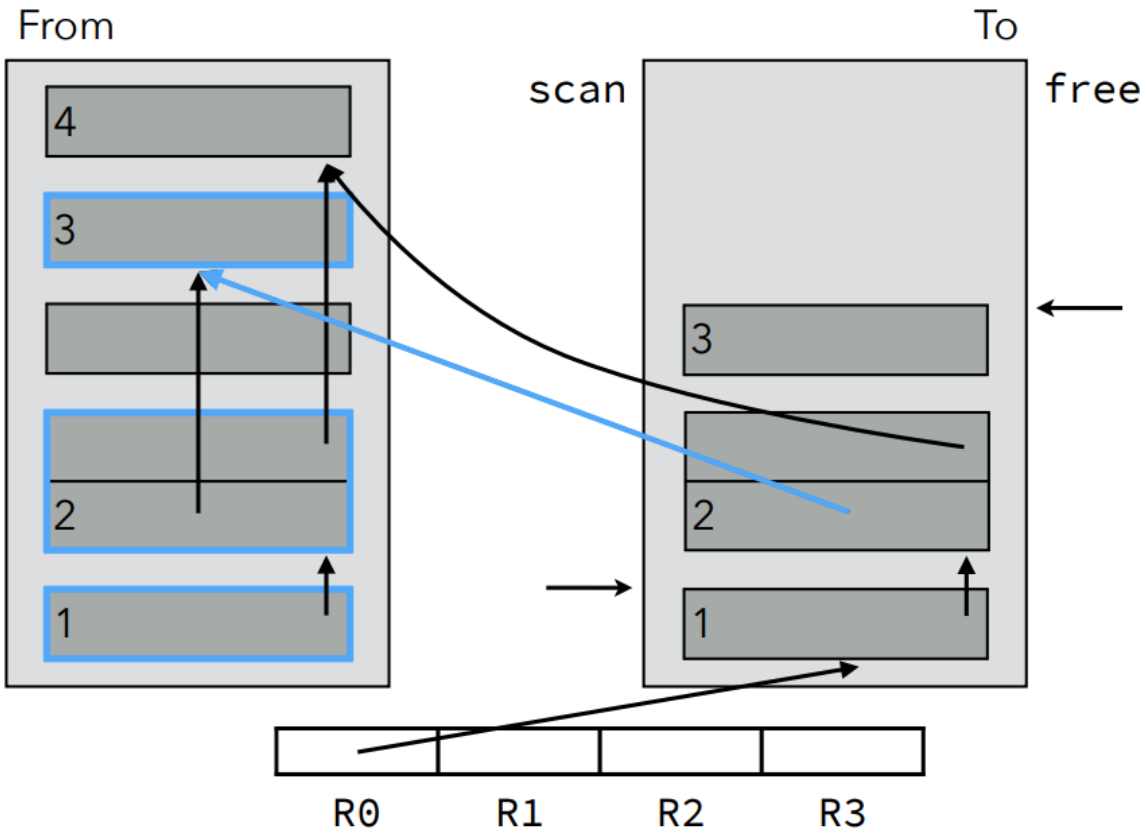
Example: Cheney's Copying GC



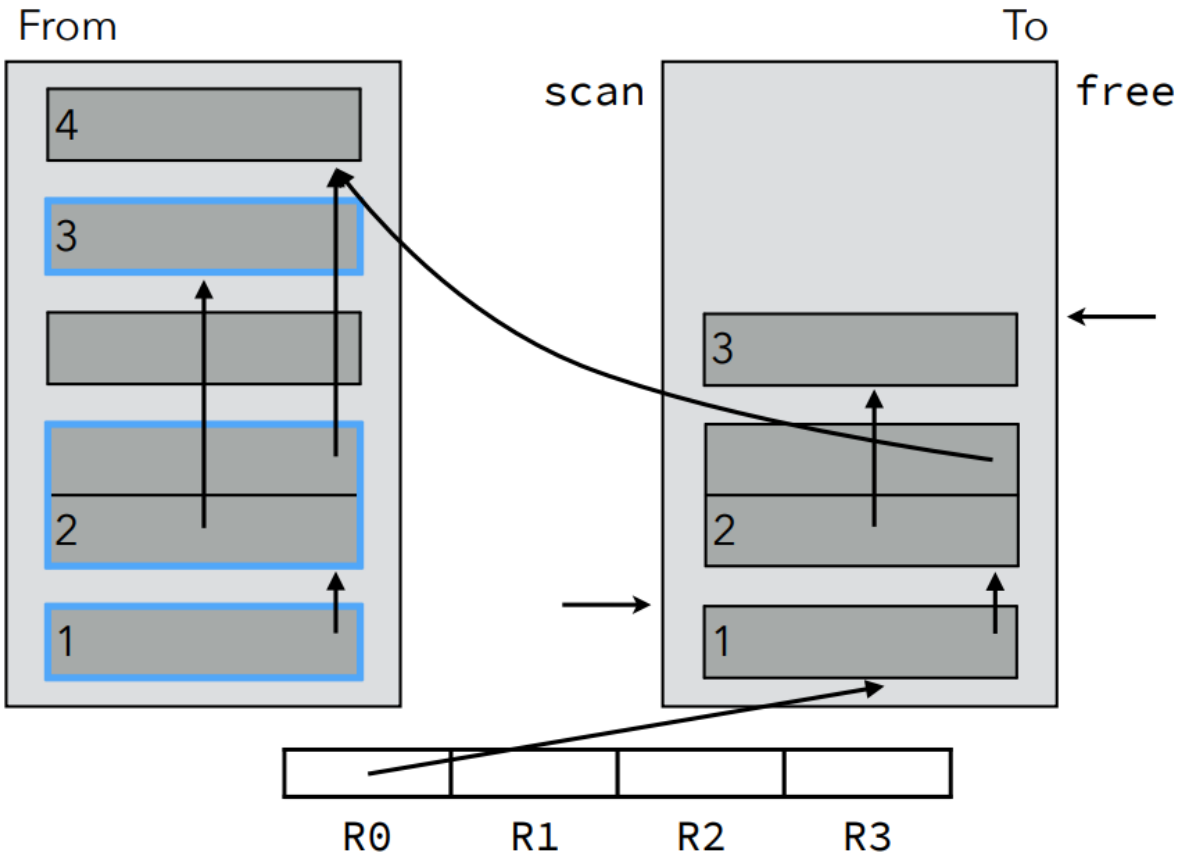
Example: Cheney's Copying GC



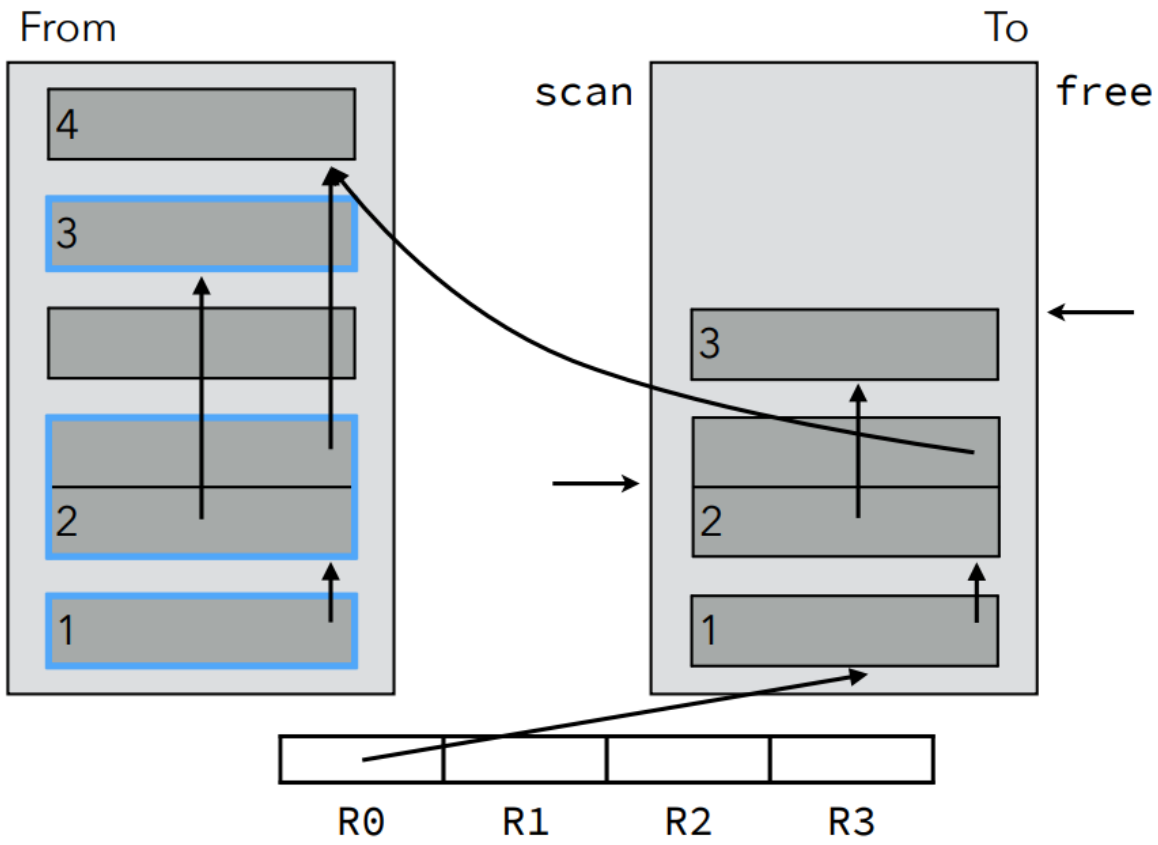
Example: Cheney's Copying GC



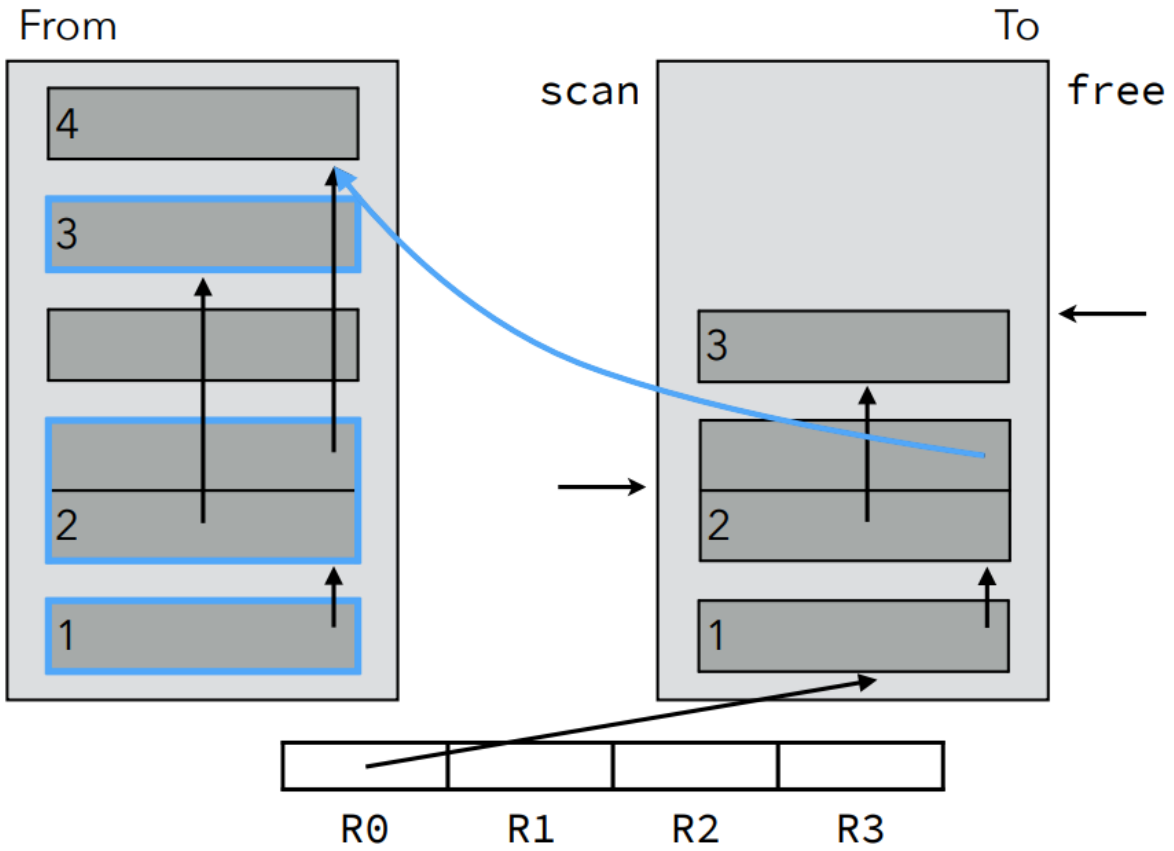
Example: Cheney's Copying GC



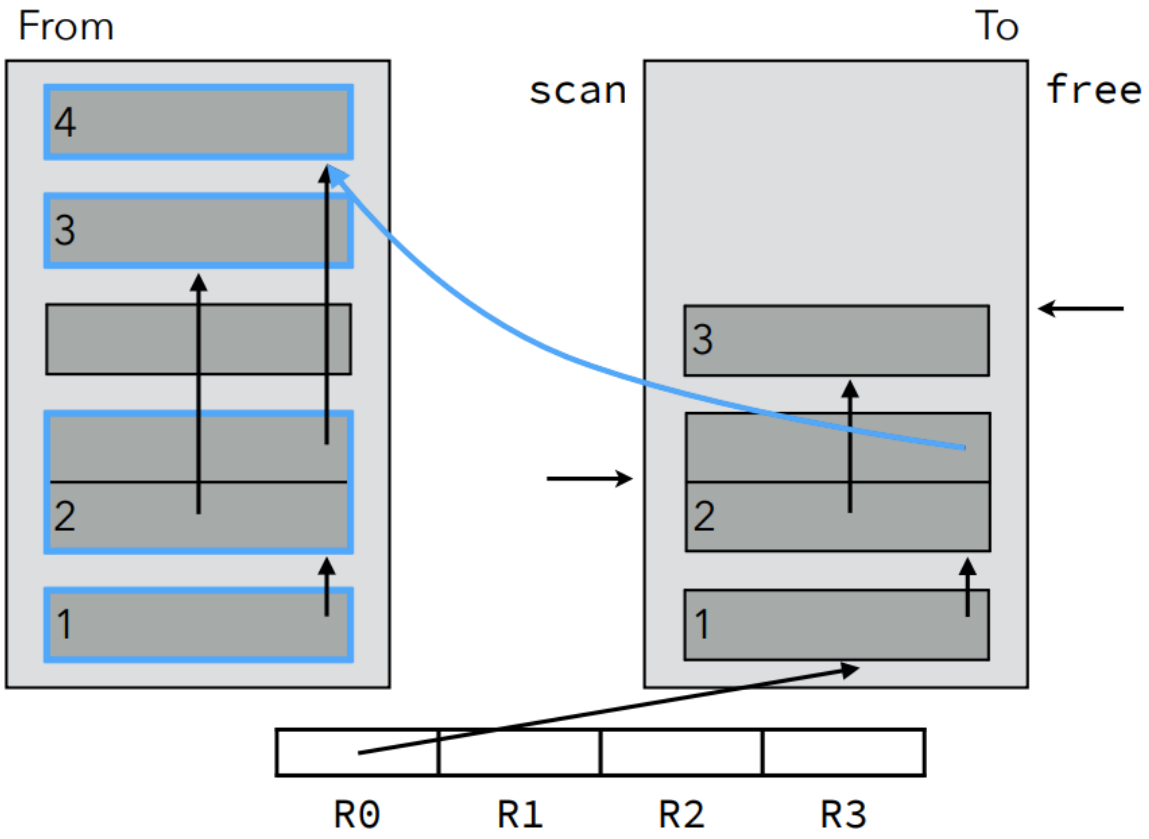
Example: Cheney's Copying GC



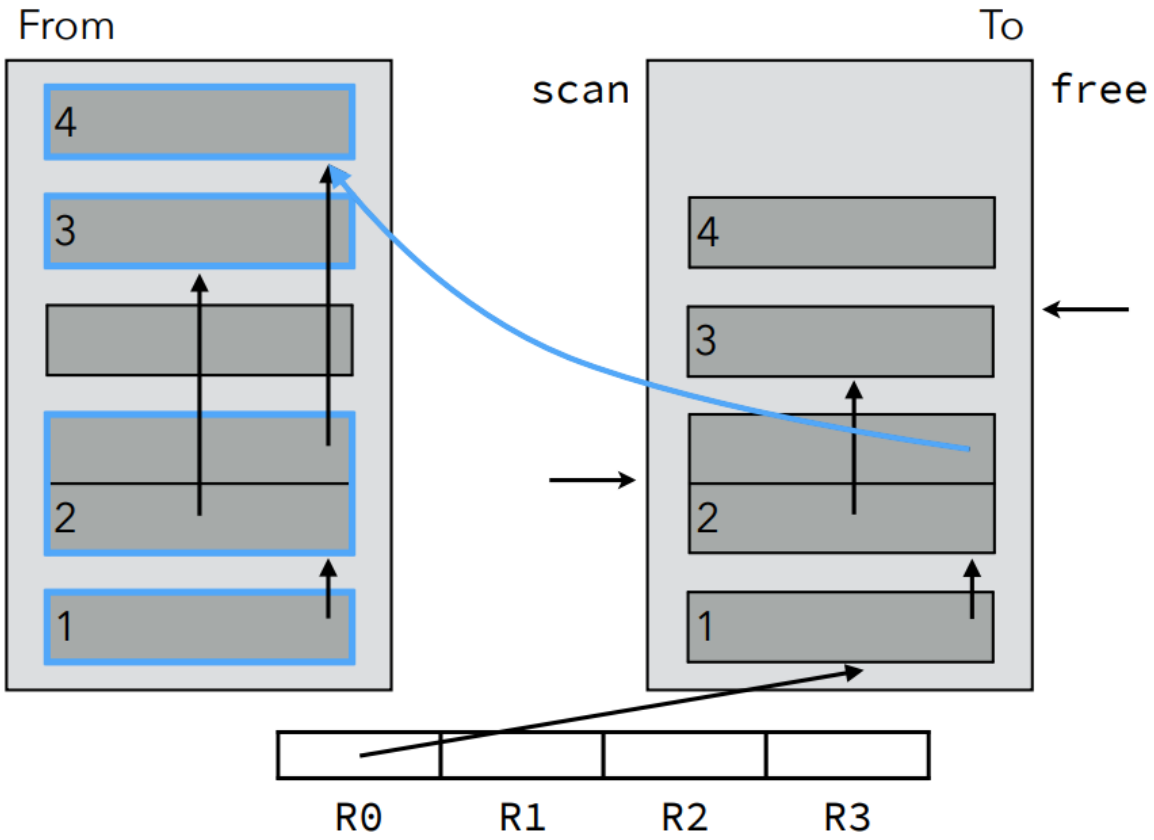
Example: Cheney's Copying GC



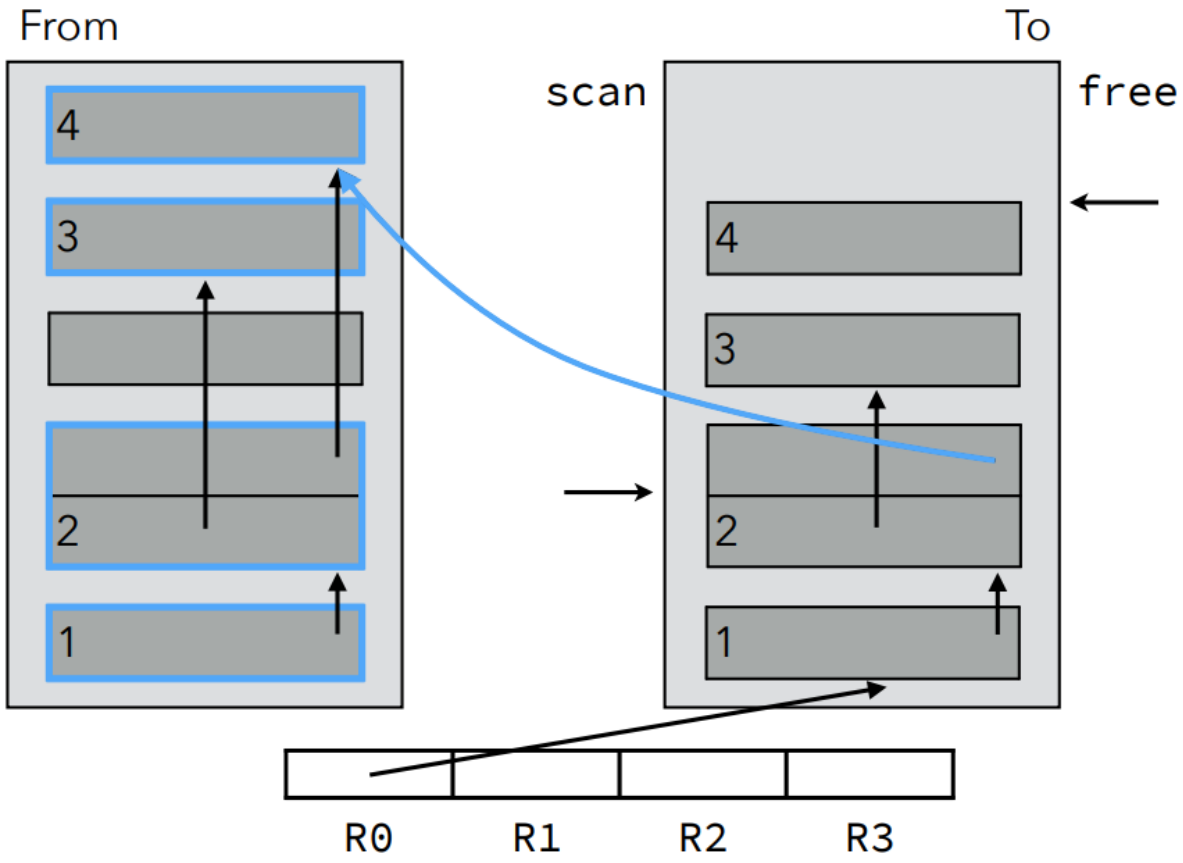
Example: Cheney's Copying GC



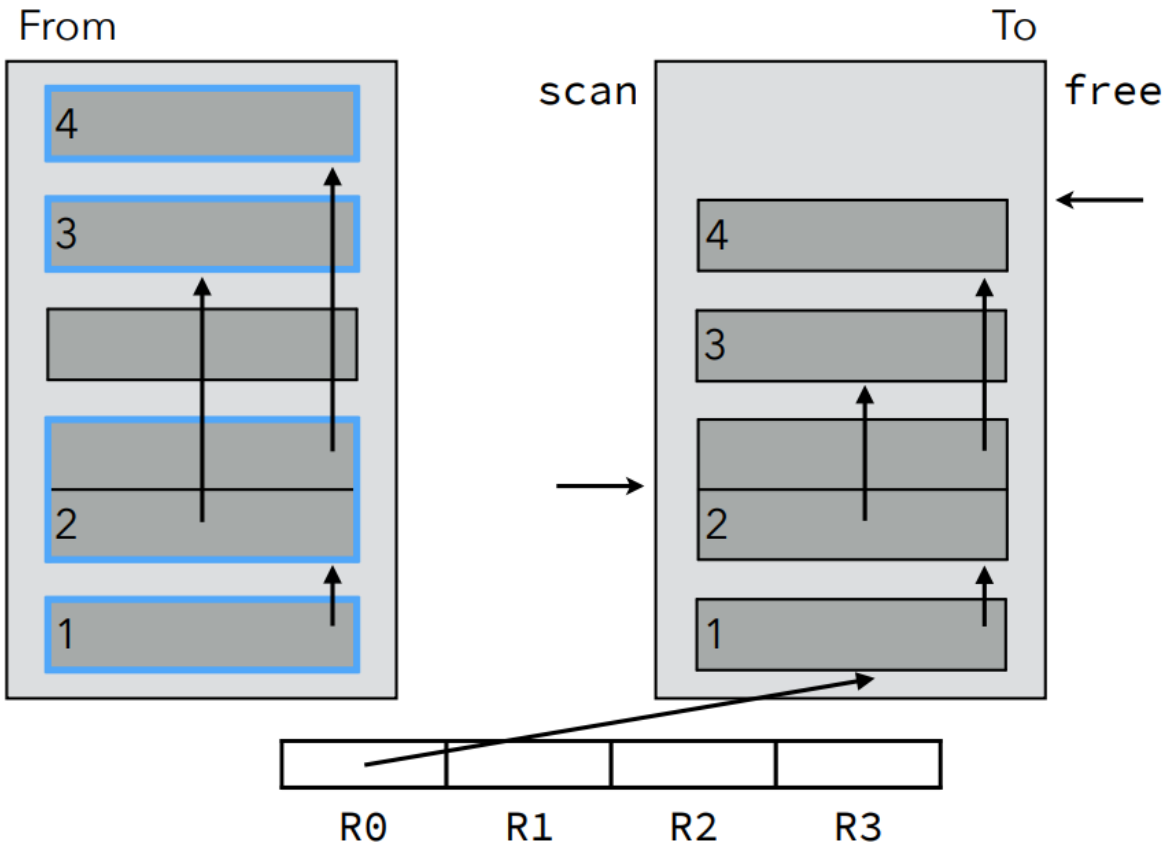
Example: Cheney's Copying GC



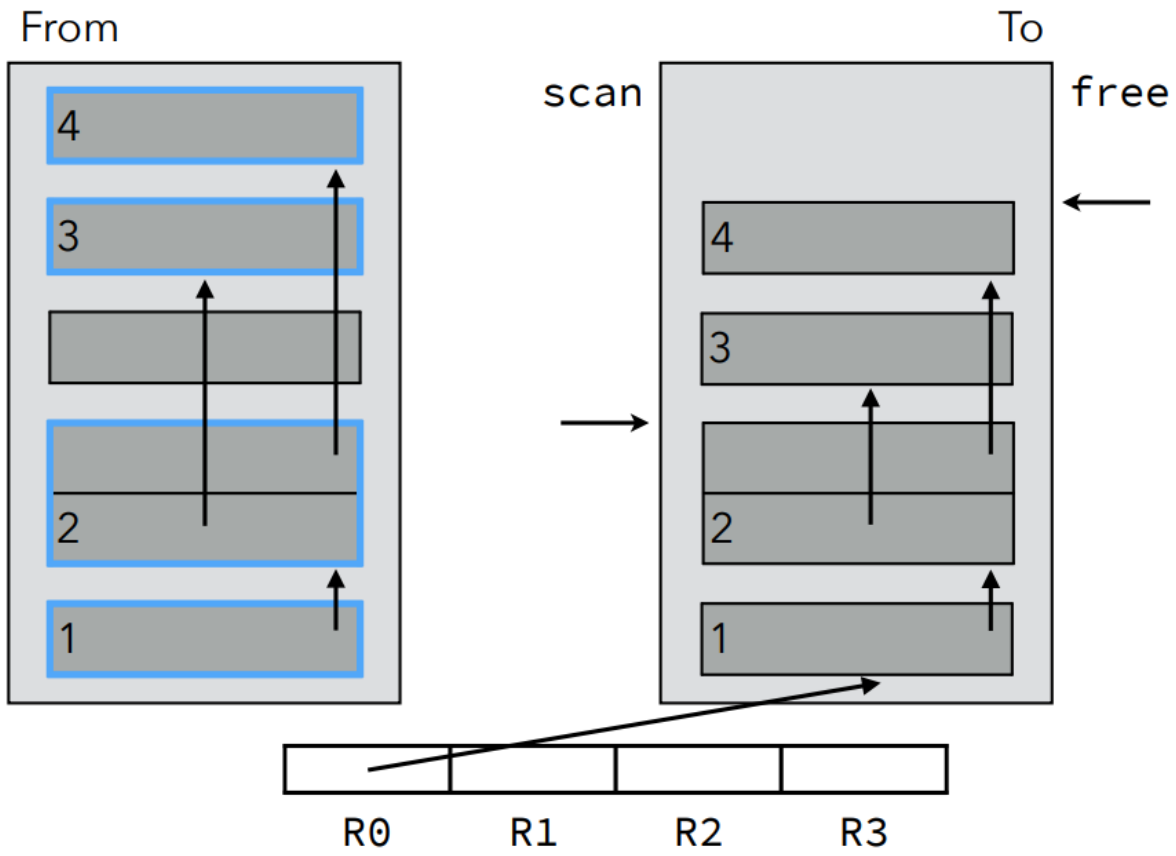
Example: Cheney's Copying GC



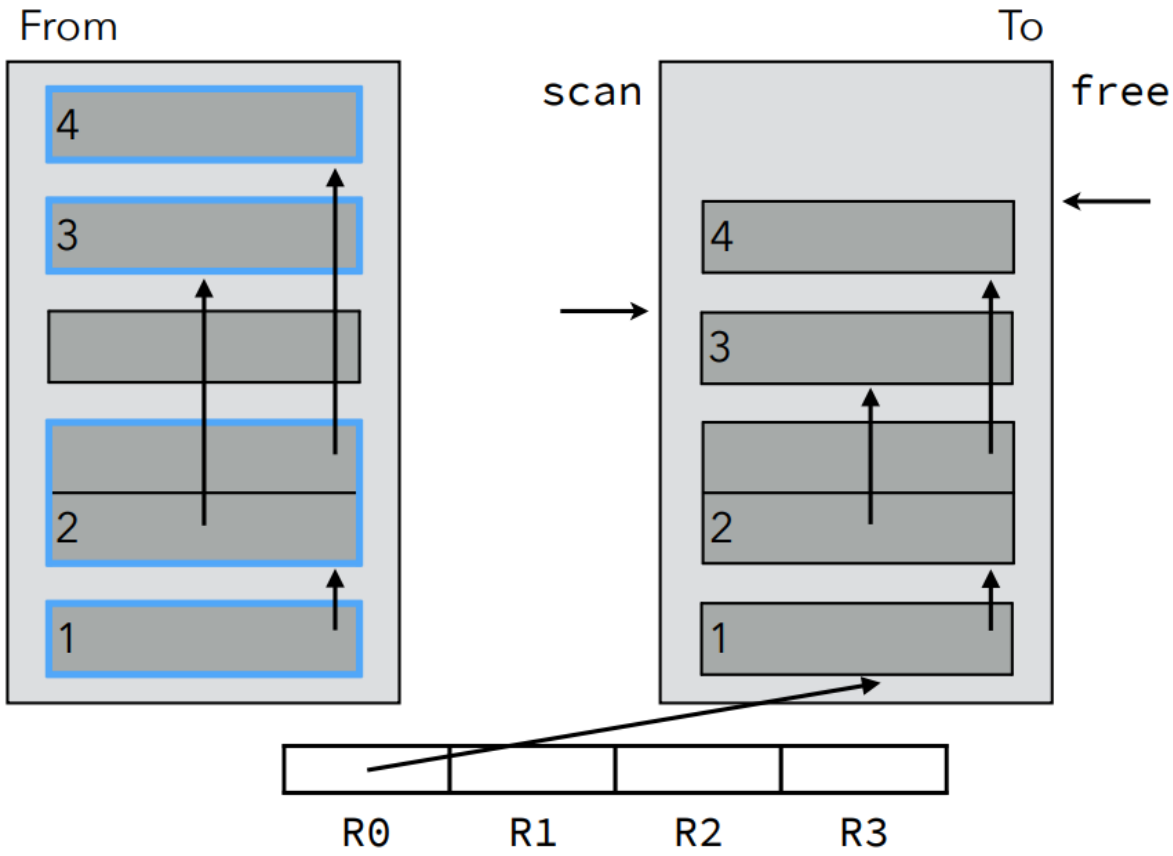
Example: Cheney's Copying GC



Example: Cheney's Copying GC



Example: Cheney's Copying GC



Example: Cheney's Copying GC